

开发日报

日期: 2026年7月2日 (星期四)

今日定位: 检索评测日 (研发计划 4.2 — 阶段4第2天 / 共19天)

组长/汇报人: 孟之语

组员: 于贺、甘毅、王佳杭

汇报时间: 2026年7月2日 22:00

1 目录

- [今日总览](#)
- [晨会记录](#)
- [组员个人进度汇报](#)
 - 3.1 孟之语 (组长/核心算法负责人)
 - 3.2 于贺 (接口与数据负责人)
 - 3.3 甘毅 (内容组织负责人)
 - 3.4 王佳杭 (评测与展示负责人)
- [今日核心工作详解](#)
 - 4.1 分段引擎回顾与质量验证
 - 4.2 评测框架架构分析
 - 4.3 Embedding 检索流程优化
 - 4.4 Smart vs Fixed 评测运行与结果分析
 - 4.5 内容组织模块评测准备
 - 4.6 参数调优框架验证
- [问题与解决方案](#)
- [晚间同步会记录](#)
- [组长总结](#)
- [关键决策记录](#)
- [明日计划](#)
- [项目整体进度追踪](#)
- [任务状态表](#)
- [里程碑验收状态](#)
- [风险管理与缓冲消耗](#)
- [附录: 关键代码与数据](#)

2 今日总览

项目	内容
日期	2026年7月2日（星期四）
阶段	阶段4 — baseline、检索评测、指标报告（第2天 / 共3天）
今日定位	检索评测日：建立向量索引，计算 Recall@k、nDCG、MRR，形成第一版指标表
里程碑	M4 — 能输出对比实验结果
全组状态	正常，全员在岗
今日计划任务	8项，完成8项
累计全组工时	约23小时
阶段4累计进度	约60%（评测准备日 + 检索评测日完成，对比实验日待进行）
上次里程碑	M3 — 核心智能体闭环初步可用（6月30日验收通过）
下次里程碑	M4 — 能输出对比实验结果（目标：7月3日）

2.1 今日在研发计划中的位置

根据研发计划（[assets/研发计划.pdf](#)）4.2节每日计划，7月2日的定位为：

检索评测日：建立向量索引，计算 Recall@k、nDCG、MRR，形成第一版指标表
负责人：王佳杭、孟之语
产出：评测脚本、指标表
里程碑：M4 — 能输出对比实验结果（baseline 与智能策略完成对比，能输出 Recall@k、nDCG、MRR）

今日是阶段4（7月1日-7月3日）的第2天。阶段4的三天行程：

日期	日程定位	核心任务	负责人	状态
7月1日	评测准备日	实现 fixed_length baseline、准备问题集和片段标注	王佳杭、全员	已完成
7月2日	检索评测日	建立向量索引，计算 Recall@k、nDCG、MRR，形成第一版指标表	王佳杭、孟之语	已完成
7月3日	对比实验日	对多策略跑评测，分析失败案例，确定最终默认策略参数	王佳杭、孟之语、甘毅	待进行

2.2 今日工作背景

在进入今日具体工作之前，先回顾项目当前的技术积累：

已完成的核心模块（全部代码在 [backend/app/services/](#) 下）：

- 分段引擎 ([segmenting/](#))：包含 parser.py（纯文本解析）、heading.py（标题识别与层级估计）、splitter.py（候选分段 + 超长拆分 + 过短合并）、segmenter.py（主流程编排）、enrichment.py（分段富化）、keyword_extraction.py（关键词提取）、statistics.py（质量统计）、models.py（数据结构定义）
- 内容组织 ([organizer/](#))：包含 organizer.py（ContentOrganizer 编排器，LLM/规则双模式）、model_client.py（RuleBasedTagger — jieba TF-IDF 关键词 + 正则实体 + 抽取式摘要 + LLMClient — DeepSeek/OpenAI 接口）
- 检索服务 ([retrieval/](#))：包含 embedding_store.py（numpy 余弦相似度检索，384维 MiniLM）、tfidf_store.py（TF-IDF char n-gram 检索）、semantic_store.py（接口兼容层）
- 评测框架 ([evaluation/](#))：包含 baseline.py（fixed_length baseline）、metrics.py（EmbeddingRelevance 相关性判定 + compute_ir_metrics 指标计算）

- 6. 预处理 (`document_preprocessor.py` + `preprocessing/cleaner.py`): 封面/目录去除、表格展平
- 7. RAG 存储 (`rag_store/`): ChromaDB 向量存储 + SQLite 元数据存储
- 8. Embedding (`embedding.py`): MiniLM 384维编码器 + 字符3-gram降级方案 + 全局单例缓存

已有的脚本工具 (`scripts/` 目录下) :

脚本	行数	功能
<code>segment_file.py</code>	—	CLI 单文件分段, 支持 txt/md/docx/pdf
<code>eval_rag.py</code>	228	RAG 评测主脚本: Smart vs Fixed 双策略对比
<code>tune_segmenting_params.py</code>	136	6维参数网格搜索, 648种组合
<code>human_eval_semantic.py</code>	299	语义完整性人工评估 (采样→CSV→打分→报告)
<code>synthesize_qa_pairs.py</code>	429	QA 对合成 (LLM 生成 + 可答性/忠实度校验)
<code>draw_architecture.py</code>	—	系统架构图生成

已有的测试 (`backend/tests/` 目录下) :

测试文件	说明
<code>test_segmenting.py</code>	分段引擎 8 个单元测试
<code>test_keyword_extraction.py</code>	关键词提取 5 个单元测试
<code>test_query_retrieval.py</code>	检索功能测试
<code>eval_dataset.py</code>	评测数据集: 3文档×5题=15题 (119行)

以上所有模块和脚本构成了今日评测工作的技术基础。

3 晨会记录 (09:30-09:55)

3.1 昨日 (7月1日) 回顾

孟之语汇报:

- 昨日花了主要时间审查现有评测框架的完整链路。逐文件阅读了 `eval_rag.py` (228行)、`baseline.py` (93行)、`metrics.py` (275行)、`embedding_store.py` (230行)、`eval_dataset.py` (119行) 五个核心文件, 确认了评测链路从文档加载到指标输出的每一步逻辑。
- 关键发现1: `eval_rag.py` 已经实现了完整的 Smart vs Fixed 双策略对比流程。`_load_and_segment()` 函数对每份文档同时运行 Smart (通过 `segment_blocks/segment_text`) 和 Fixed (通过 `fixed_length_segment`) 两种分段, 然后分别构建 `EmbeddingStore` 索引进行检索对比。这个流程可以直接用于今天的评测工作, 不需要额外开发。
- 关键发现2: `metrics.py` 中的 `EmbeddingRelevance` 类实现了混合相关性判定——关键词命中率 $\geq 34\%$ 直接判定为相关, 否则以 `embedding` 余弦相似度 (阈值 0.45) 判定, 同时对于包含数字/指标证据的 `chunk` 给予一定的宽松处理 (`contains_number_or_metric` 函数)。这个混合方案比纯 `embedding` 或纯关键词匹配更鲁棒。
- 关键发现3: `baseline.py` 仅实现了 `fixed_length_segment` (512字符窗口, 在最近句子边界处切分), 并没有实现 `heading_based` `baseline`。按照研发计划, 我们至少需要比较三种策略 (`fixed_length`、`heading_based`、`structure_semantic_hybrid`)。`heading_based` 的缺失需要在后续计划中补齐。
- 关键发现4: `tune_segmenting_params.py` (136行) 已经定义了一个 6 维的搜索空间——`min_chars` [220,260,300]、`target_chars` [650,750,900]、`max_chars` [900,1000,1200]、`heading_flush_min_chars` [120,180,240,300]、`semantic_boundary_threshold` [0.35,0.45,0.55]、`overlap_sentences` [0,1,2]。总共 648 种组合 (经过 `iter_configs()` 过滤无效组合后约 400+ 可用)。这个搜索框架可以直接用于后天的对比实验, 不需要重写。
- 整理了阶段3 (6月28-30日) 的技术总结, 为最终技术报告积累素材。阶段3完成了语义边界评分算法 (基于

sentence-transformers MiniLM 的余弦相似度)、5类特殊块保护(表格/公式/代码/列表/图片)、RuleBasedTagger 规则内容组织、LLMClient DeepSeek 增强、ContentOrganizer 编排器、文档级摘要聚合。这些内容可以作为技术报告中"核心算法设计"章节的素材。

- 发现 eval_dataset.py 的 EvalQuestion 数据结构只有 question 和 answer_keywords 两个字段,缺少问题类型(定义类/流程类/指标类等)和难度等级的标注。如果要做分层分析,需要手动分类或扩展数据结构。

于贺汇报:

- 昨日梳理了评测框架中 embedding 计算链路的性能特征。EmbeddingEncoder(embedding.py 第 38-85 行)使用 get_default_encoder() 获取全局单例,首次调用时加载 MiniLM 模型(118MB, 384维),耗时约 15 秒。后续调用复用同一实例,不再重复加载。
- 检查了 embedding_store.py 中 add_chunks() 方法的实现(第 50-69 行):它先对每个 chunk 调用 _enrich_text() 生成增强文本(标题路径+正文),然后调用 self._encoder.encode(texts) 进行批量编码。这里有一个可以优化的点: _enrich_text() 的逻辑比较简单(只是字符串拼接),可以考虑在检索时也使用相同的 enrich 逻辑,确保索引和查询的文本表示一致。
- 仔细阅读了 statistics.py(133行)的四个统计函数: build_statistics() 计算 chunk 数量和平均长度、 ends_at_sentence_boundary() 检查 chunk 是否在合法句子边界结束、 is_protected_block_intact() 检查受保护块是否完整、 count_tokens() 进行轻量 token 估算(中文字符+英文单词+标点)。这四项逻辑完整、边界处理得当,可以直接用于今天的质量验证。
- 编写了 1 个 EmbeddingStore 索引构建的单元测试,验证了 add_chunks → search 的基本链路正常。

甘毅汇报:

- 昨日重点研究了内容组织模块的代码结构。organizer.py(约 120 行)定义了 ContentOrganizer 类和 OrganizeResult 数据结构。其核心逻辑是: organize_chunk() 方法先尝试 LLM 模式(如果 self._llm.is_available),失败则降级到规则模式(self._rule_organize())。 organize_batch() 方法支持批量处理多个 chunk。
- model_client.py(约 200+ 行)定义了 RuleBasedTagger 类和 LLMClient 类。RuleBasedTagger 实现了三个核心方法: extract_tags() (jieba 分词 → 停用词过滤 → TF-IDF 权重 → bigram 组合 → 输出 top-k 标签)、 generate_summary() (Lead-1 首句 + TF-IDF 句子打分,截断 ≤ 80 字)、 extract_entities() (6 种正则模式:百分比/指标/日期/版本号/机构/阈值)。
- 研究了内容组织模块的评测方法。现有的 evaluator.py(在 backend/app/services/ 下)提供了标签/摘要/实体质量的评估逻辑,需要了解其评测入口和输出格式。
- 开始设计内容组织质量的人工标注方案。计划从 3 篇文档中各抽取 5 个 chunk(共 15 个),人工标注标准关键词列表和标准摘要文本,作为 ground truth 用于评测 RuleBasedTagger 的输出质量。
- 发现了一个方法学问题:规则生成的标签(如"语义边界")和人工标注的标签(人工可能写成"语义相似度边界计算")在字符串层面天然不同,精确匹配会严重低估准确率。需要采用词级 Jaccard 重叠作为评测指标。这个方案与 6 月 26 日 P0 评估方法论修复中使用词级重叠评测标签的思路一致。

王佳杭汇报:

- 昨日完成了评测框架的全面梳理和初次运行。具体工作包括:
 - 逐行阅读了 eval_rag.py 的完整代码(228 行),理解了 load_and_segment() → run_evaluation() → print_summary() 的三段式评测流程。
 - 确认 eval_rag.py 使用的检索后端是 EmbeddingStore(embedding_store.py),而不是 ChromaDB。EmbeddingStore 是纯内存的 numpy 余弦相似度检索引擎,每次评测都需要重新构建索引。这对于 3 文档 × 约 40 个 chunk 的规模完全够用(构建索引 < 2 秒),但如果后续文档数增加到 10+ 或每个文档 chunk 数增加到 100+,可能需要考虑索引持久化。
 - 初步运行了一次 Smart vs Fixed 全量评测(3 文档 × 5 题 = 15 题),得到的指标与 README.md 中记录的结果一致: Smart Recall@5 = 0.503、Fixed Recall@5 = 0.576。差距 -12.7%。

- 逐题查看了 Smart Recall@5 和 Fixed Recall@5 的对比。15 题中，Smart 胜 5 题、平 3 题、负 7 题。胜负比 5:7，差距并不悬殊。
- 检查了 tune_segmenting_params.py 的代码（136 行），确认了搜索空间定义、`iter_configs()` 配置生成器、`evaluate_config()` 单配置评测函数、`rank_key()` 排序函数均工作正常。搜索空间共 6 维 648 种组合，但当前 target_chars 的候选值最大只到 900，没有覆盖 1000-1200 的范围。如果需要调大 target_chars（针对 Smart chunk 平均长度偏短的问题），需要手动增加候选值。

3.2 昨日全天工作总结

成员	主要产出	工时
孟之语	评测框架 5 文件审查、阶段3 技术总结、heading_based 缺失发现、数据集结构梳理	约 5.5h
于贺	Embedding 链路性能分析、statistics.py 审查、单元测试 1 个	约 5h
甘毅	organizer.py/model_client.py 代码审查、评测方案初设、标注方案设计	约 5h
王佳杭	eval_rag.py 全量初跑、tune 脚本验证、per-question 对比分析	约 6h

3.3 昨日遗留问题

编号	问题描述	影响范围	责任人	状态
P-7.1-1	Smart vs Baseline Recall@5 差距约 -12.7%，距离 +10% 目标有 22.7pp 差距	核心评测指标	孟之语、王佳杭	今日重点攻关
P-7.1-2	heading_based baseline 尚未实现，三策略对比缺一个维度	评测完整性	孟之语	列入后续计划
P-7.1-3	评测数据集缺少问题类型和难度标注字段	分层分析能力	孟之语	今日手动分类
P-7.1-4	tune 搜索空间 target_chars 最大仅 900，未覆盖更大候选值	参数调优	王佳杭	对比实验日前扩大
P-7.1-5	内容组织质量评测尚未启动，无 ground truth 数据	内容组织验收	甘毅	今日启动标注

3.4 今日任务分配

编号	任务	负责人	预计工时	优先级	依赖
T-8.1	构建 EmbeddingStore 向量索引：对 3 文档的 Smart 和 Fixed 分段结果分别构建 numpy 索引	王佳杭	2h	P0	—
T-8.2	全量运行 eval_rag.py，计算 Recall@1/3/5、Precision@5、nDCG@5、MRR，输出 per-question 对比	王佳杭	2h	P0	T-8.1
T-8.3	形成 Smart vs Fixed 对比指标表：按文档、按问题类型、整体汇总三个维度	王佳杭、孟之语	2h	P0	T-8.2
T-8.4	分析 Smart 策略 Recall@5 低于 Baseline 的根因：chunk 长度分布、语义阈值敏感性、overlap 效果	孟之语	3h	P0	T-8.3
T-8.5	EmbeddingStore 索引构建性能优化：批量编码、确认单例缓存效果	于贺	2h	P1	T-8.1
T-8.6	分段质量统计指标验证：对 3 份文档运行 statistics.py，确认四项指标达标	于贺	2h	P1	—
T-8.7	内容组织评测方案详细设计 + 人工标注启动：15 chunk 标注（关键词 + 摘要）	甘毅	4h	P1	—
T-8.8	评测数据集问题类型手动分类 + 扩充方案讨论	甘毅、王佳杭	1h	P2	—

3.5 晨会关键决策

决策 1 — 评测策略确定

背景：研发计划要求至少比较三种策略（fixed_length、heading_based、structure_semantic_hybrid）。当前 baseline.py 仅实现了 fixed_length，heading_based 尚未实现。

讨论过程：

- 孟之语提出，heading_based baseline 的实现工作量不大（利用已有的 heading.py 标题识别模块，按标题层级切分 + 长度控制），估计 2-3 小时可完成。但今天的主要精力应该放在 Smart vs Fixed 的深度分析上，因为这是当前已知的差距所在。如果今天分散精力去实现 heading_based，可能两头都做不深。
- 王佳杭同意，认为当前最重要的是理解 Recall@5 差距的根因，然后再扩展评测维度。
- 于贺建议，heading_based 可以在明天对比实验日和参数调优一并完成，届时可以将 heading_based 也纳入网格搜索的评测对象。

决议：今日评测以 Smart vs Fixed-length 双策略对比为核心。heading_based baseline 推迟到 7 月 3 日（对比实验日）实现。当前 eval_rag.py 已支持双策略对比逻辑，可直接使用。

决策 2 — Recall@5 根因分析优先级

背景：当前 Smart Recall@5 = 0.503，Fixed Recall@5 = 0.576，差距 -12.7%。距离 +10% 目标有 22.7 个百分点的差距。如果明天要进行参数调优，需要先明确调整方向，避免盲目搜索。

讨论过程：

- 孟之语提出三个假设方向：(1) chunk 长度偏短导致信息密度不足；(2) semantic_boundary_threshold=0.45 过于敏感导致过度切分；(3) overlap_sentences=1 上下文窗口不足。
- 甘毅补充了一个方向：retrieval_text 中标题路径的权重是否足够？如果 chunk 的 embedding 主要由正文决定而标题信息被稀释，结构感知的优势就无法在检索中体现。
- 王佳杭认为可以先用 per-question 结果来验证假设。如果长度假设成立，那么在宽泛查询（需要长文本覆盖的问题）上 Fixed 应该明显领先；如果语义阈值假设成立，那么在主题过渡段落的问题上 Smart 应该表现较差。

决议：孟之语今日集中精力做根因分析。从三个角度入手：(1) 统计 Smart chunk 和 Fixed chunk 的长度分布，计算信息密度差异；(2) 分析语义边界触发的频率和位置，评估是否存在过度切分；(3) 评估 overlap 和 retrieval_text 的实际贡献。

决策 3 — 内容组织评测方法

背景：甘毅指出规则生成的标签和人工标注的标签在表述上天然不同（如规则生成“语义边界”，人工可能标注“语义相似度边界计算”），精确字符串匹配会严重低估准确率。

讨论过程：

- 甘毅提议采用词级 Jaccard 重叠：将规则标签和人工标注标签分别分词后，计算 |交集| / |并集|。阈值设为 0.5（即一半以上的词重叠即视为正确）。
- 孟之语确认这个方案与 6 月 26 日 P0 评估方法论修复中的词级重叠方案一致，可以保持评测方法论的前后一致性。
- 于贺建议后续如果接入 LLM-as-judge，可以让 LLM 直接判断“规则标签是否与人工标注标签语义等价”，作为词级 Jaccard 的补充。

决议：标签准确率评测采用词级 Jaccard 重叠（阈值 0.5）。摘要忠实度采用人工判定（摘要中的陈述是否能在原文中找到对应）。实体精度采用精确匹配 + 类型校验。后续可扩展 LLM-as-judge 辅助评测。

决策 4 — 评测数据集

背景：当前 `eval_dataset.py` 包含 3 文档 × 5 题 = 15 题。问题类型分布未标注，数据集缺少难度分级。

讨论过程：

- 王佳杭认为 15 题规模偏小，单个问题的随机波动（如标注偏差、文档格式问题）对文档级指标的影响可达 5-10 个百分点。建议扩充到每文档 10-15 题。
- 甘毅补充，如果明天要做参数网格搜索（可能涉及 50+ 次全量评测），15 题的评测成本低（每次约 15-20 秒），是有利的一面。但如果搜索出的最优参数过拟合到这 15 题上，评测结论就不可靠。
- 孟之语折中：今天先用 15 题产出第一版指标表，验证评测链路通畅。明天（对比实验日）上午由甘毅和王佳杭为每文档新增 5 题（扩至 $3 \times 10 = 30$ 题），然后再做参数搜索。

决议：今日以现有 15 题为准产出第一版指标表。明天上午扩充至 30 题后，再进行参数网格搜索。

决策 5 — 评测数据输出格式

背景：`eval_rag.py` 当前的输出格式是终端友好的表格 + 汇总行，但没有结构化的 JSON 输出，不便于后续的数据分析和报告嵌入。

讨论过程：

- 于贺建议增加 `--output-json` 参数，将评测结果输出为结构化 JSON 文件。
- 孟之语认为这个改进优先级不高，今天可以手动整理指标表，后续（集成测试阶段）再统一改进输出格式。

决议：今天以手动整理 + 终端输出为主。JSON 输出格式改进留到集成测试阶段（7月4日-6日）。

4 组员个人进度汇报

4.1 孟之语（组长 / 核心算法负责人）

今日完成：

1. [T-8.3] 与王佳杭合作完成 Smart vs Fixed 对比指标表，按文档和整体两个维度汇总 ✓
2. [T-8.4] Recall@5 根因分析完成，定位到 3 个关键影响因素 ✓
3. 设计了分段参数调优的三阶段方案 ✓
4. 手动分类了 15 个问题的类型（定义类/流程类/指标类/对比类/细节定位类），输出了分类统计 ✓
5. 梳理了 `tune_segmenting_params.py` 的搜索空间，提出扩大 `target_chars` 候选值的调整方案 ✓
6. 开始撰写技术报告的"核心算法设计"和"评测与对比实验"章节大纲 ✓
7. 晚间同步会上向全组汇报了根因分析结果和参数调整建议 ✓

实际工时：约 6.5 小时

关键产出详情：

产出 1 — Recall@5 根因分析

通过以下三个维度的分析，定位到 Smart 策略 Recall@5 低于 Fixed Baseline 的 3 个根因：

维度一：Chunk 长度分布分析

对 3 份文档的 Smart 分段和 Fixed 分段输出进行了长度统计：

文档	Smart chunk 数	Smart 平均长度	Fixed chunk 数	Fixed 平均长度
title.md	4	~856 字符	5	~513 字符
调研报告.docx	~12	~467 字符	~14	~514 字符
开题报告.docx	~15	~493 字符	~16	~512 字符

Smart 在 title.md（只有 4 个一级标题的结构简单文档）上反而 chunk 更长，因为每个标题下都是一段完整的段落。但在两个 DOCX 文档（结构更复杂、包含更多子标题和短段落）上，Smart chunk 平均长度在 470-500 字符左右，比 Fixed 的 512-514 字符短约 3-8%。这个差距看似不大，但在检索场景中，Fixed 的每个 chunk 恰好是 512 字符（在句子边界对齐后略超），信息密度略高。当查询较宽泛时（如“文档的主要内容是什么”、“系统采用了什么技术方案”），Fixed 长 chunk 单次检索命中就能覆盖更多相关信息，而 Smart 的较短 chunk 需要命中多个才能覆盖相同的语义空间。

维度二：语义边界触发频率分析

通过分析 splitter.py 中 `build_candidate_chunks()` 函数的逻辑（第 22-189 行），发现语义边界触发条件包含在 `flush_current("semantic_boundary")` 调用中。当前 `semantic_boundary_threshold=0.45` 意味着：当相邻两个段落的 embedding 余弦相似度低于 0.45 时，就在两者之间切分。阈值 0.45 是一个相对敏感的设定——段落间存在 mild 主题转移（如从“系统架构概述”过渡到“各模块详细说明”）时，相似度可能在 0.4-0.5 之间，从而触发切分。这导致语义连贯的相邻段落被拆散到不同 chunk 中，用户在查询“系统架构”时，详细介绍模块的部分（虽然在逻辑上属于同一章节）可能因为 0.45 的阈值被分到了下一个 chunk。

维度三：overlap 和 retrieval_text 分析

当前 `overlap_sentences=1`，意味着每个 chunk 仅与前一个 chunk 共享 1 个句子。在文档内容高度连续时（如学术论文的方法章节，连续的多个段落都在描述同一个方法的步骤），1 句 overlap 提供的上下文覆盖非常有限。相邻 chunk 之间的大部分信息仍然是互斥的。

retrieval_text 方面，embedding_store.py 的 `enrich_text()` 函数（第 196-216 行）将标题路径拼接接到 chunk 内容之前。但标题路径通常在 20-50 字符左右，而正文内容在 400-500 字符左右，标题信息的权重在 embedding 编码中占比不到 10%。这意味着结构感知的信息在检索向量中被稀释了。

根因总结：

根因编号	描述	影响估测	调整方向
RC-1	Smart chunk 平均长度偏短，信息密度略低于 Fixed 512 字符窗口	主因（影响约 7-8 题）	target_chars 从 900 → 1000-1100
RC-2	semantic_boundary_threshold=0.45 偏敏感，mild 主题转移即触发切分	次因（影响约 4-5 题）	threshold 从 0.45 → 0.50-0.55
RC-3	overlap_sentences=1 上下文窗口小 + 标题路径在 retrieval_text 中权重不足	辅因（影响约 3-4 题）	overlap 从 1 → 2-3；retrieval_text 增加标题权重

这三个根因的调整方向是协同的：增大 target_chars 会让每个 chunk 更长，减少 chunk 总数；提高 semantic_threshold 会减少切分点，进一步增加 chunk 长度；增加 overlap 会提升相邻 chunk 的信息覆盖重叠。三者叠加预计能够将 Smart Recall@5 从 0.503 提升至 0.55-0.60 区间（目标 Baseline=0.576）。

产出 2 — 15 题问题类型手动分类

问题类型	题数	占比	定义
定义类	4	26.7%	询问概念定义、系统/课题是什么
流程类	2	13.3%	询问操作步骤、处理流程
指标类	3	20.0%	询问具体数值、指标要求
对比类	3	20.0%	询问差异、对比、区别
细节定位类	3	20.0%	询问具体细节、特定段落内容

当前 15 题的类型分布不均衡（流程类仅 2 题），扩充到 30 题后需要确保每种类型至少 5 题。

产出 3 — 技术报告大纲（初稿）

1	技术报告大纲
2	
3	1. 课题背景与目标
4	1.1 课题定位
5	1.2 任务书要求拆解（10项能力）
6	1.3 交付物清单
7	
8	2. 系统架构设计
9	2.1 总体架构（文档加载→预处理→分段→内容组织→RAG存储→API→前端）
10	2.2 数据结构设计（DocumentBlock、Chunk、SegmentConfig）
11	2.3 API 接口设计（/health、/segment、/organize、/evaluate、/strategies）
12	2.4 技术栈选型与降级策略
13	
14	3. 核心算法设计
15	3.1 分段流程总览（标题候选块→超长拆分→过短合并→finalize）
16	3.2 标题边界优先策略
17	3.3 语义边界增强（embedding 相似度 + 边界评分）
18	3.4 特殊块保护（表格/公式/代码/列表/图片）
19	3.5 长度控制与 overlap 机制
20	3.6 内容组织（关键词/摘要/实体/标签）
21	3.7 回链与元数据（title_path、source_refs、strategy_info、quality_flags）
22	
23	4. 评测与对比实验
24	4.1 评测框架设计（文档→分段→索引→检索→相关性判定→指标计算）
25	4.2 Baseline 策略（fixed_length、heading_based）
26	4.3 评测数据集（文档选择、问题设计、标注方案）
27	4.4 IR 指标（Recall@k、Precision@k、nDCG@k、MRR）
28	4.5 分段质量统计（不破句率、表格成块率、长度命中率、回链完整率）
29	4.6 内容组织质量评测（标签准确率、摘要忠实度、实体精度）
30	4.7 Smart vs Baseline 对比结果与分析
31	4.8 参数调优实验与最优配置
32	
33	5. 失败案例分析
34	5.1 Badcase 分类与统计
35	5.2 典型失败模式分析
36	5.3 改进方向与后续工作
37	
38	6. 工程实现
39	6.1 服务集成（FastAPI + Uvicorn）
40	6.2 前端展示（Vue 3 + Vite）
41	6.3 CLI 工具与脚本
42	6.4 部署说明
43	
44	→ 后续内容

45	7.1 课题完成情况
46	7.2 技术亮点与创新
47	7.3 已知限制
48	7.4 改进方向

遇到的阻塞：

- 无重大阻塞。在 per-question 分析中发现 eval_dataset.py 中有 3 个问题的 answer_keywords 包含英文缩写（如"RAGFlow"、"Docling"），而实际 chunk 中可能使用中文全称描述同一概念。由于 EmbeddingRelevance 的 keyword_score() 使用 normalize_for_keyword_match()（去除空格 + 小写化），英文缩写匹配不受影响，但中文全称 vs 英文缩写的不匹配会导致 keyword_score 降低。已标注这 3 个问题为需要检查标注一致性的项目，不影响今日指标表产出。

4.2 于贺（接口与数据负责人）

今日完成：

- [T-8.5] EmbeddingStore 索引构建性能优化完成：确认单例缓存效果、优化批量编码逻辑 ✓
- [T-8.6] 分段质量统计指标验证完成：对 3 份文档运行 statistics.py，4 项核心指标全部达标 ✓
- 编写了 1 个 EmbeddingStore 索引构建与检索的集成测试 ✓
- 确认了 embedding.py 中降级机制的完整性：sentence-transformers 可用时使用 MiniLM（384维），不可用时自动降级到字符 3-gram 哈希向量（256维） ✓
- 为 eval_rag.py 的运行流程做了性能计时分析，确认了各环节耗时占比 ✓
- 梳理了当前项目中的重复和可整合模块：rag_store/（ChromaDB + SQLite）和 retrieval/（EmbeddingStore + TF-IDF + SemanticStore）存在功能重叠 ✓

实际工时：约 5.5 小时

关键产出详情：

产出 1 — EmbeddingStore 性能分析

对 eval_rag.py 的一次完整评测运行进行了各环节耗时分析：

环节	耗时	占比	说明
文档加载 + 分段	~3s	16%	3份文档，Smart + Fixed 各分段一次
Embedding 编码	~6s	32%	~80个chunk × 2策略 = ~160次编码（批量处理）
索引构建	~0.5s	3%	numpy 矩阵构建，非常快
检索（15题 × 2策略）	~3s	16%	30次检索，每次 cosine 相似度计算
相关性判定	~5s	26%	每次检索对 top-5 chunk 计算 keyword_score + embedding 相似度
指标计算 + 输出	~1.5s	8%	IR 指标计算 + 终端打印
总计	~19s	100%	

主要耗时在 embedding 编码（32%）和相关性判定（26%）。Embedding 编码的瓶颈是 MiniLM 模型推理，相关性判定的瓶颈是对每个 chunk 单独计算 embedding 相似度（judge.score(chunk_content) 内部调用了 self._encoder.encode([chunk_content])，每次调用都会触发一次模型推理）。

优化建议（已记录，待后续实现）：

- 相关性判定改为批量编码：将 top-k 的所有 chunk 一次性编码，而非逐条编码。预估将相关性判定耗时从 5s 降至 2s。

- Embedding 编码可以预计算并缓存到磁盘：首次编码后将向量保存到 `.npy` 文件，后续评测直接加载，省去模型推理时间。

当前总耗时 19s 对于开发调试来说可接受，不需要立即优化。后续如果评测数据集扩充至 100+ 题，再考虑实施上述优化。

产出 2 — 分段质量统计验证

使用 `scripts/segment_file.py` 对 3 份文档分别运行分段（使用默认 `SegmentConfig`），然后调用 `statistics.py` 的 `build_statistics()` 函数验证统计指标。

以 `title.md` 为例的详细验证过程：

```
1 1. 加载文档: assets/title.md (纯文本/Markdown)
2 2. 解析为纯文本块: 28 个 DocumentBlock
3 3. 构建候选 chunk: 4 个 CandidateChunk
4 4. 超长拆分: 0 个需要拆分
5 5. 过短合并: 0 个需要合并
6 6. 最终 Chunk 列表: 4 个 Chunk
7 7. 调用 build_statistics(chunks, config):
8   - chunk_count: 4
9   - avg_chars: ~856
10  - target_length_hit_rate: 4/4 = 100%
11  - source_ref_complete_rate: 4/4 = 100%
12  - no_break_sentence_rate: 4/4 = 100%
13  - table_code_formula_intact_rate: 4/4 = 100% (无特殊块)
```

三份文档的汇总结果：

文档	不破句率	表格成块率	长度命中率	回链完整率	chunk数
title.md	100%	100%	100%	100%	4
调研报告.docx	100%	100%	91.7%	100%	~12
开题报告.docx	100%	100%	93.3%	100%	~15

指标	目标	3文档加权平均	判定
不破句率	100%	100%	达标
表格/公式/代码整体成块率	≥95%	100%	达标
目标长度区间命中率	≥90%	94.4%	达标
原文回链完整率	100%	100%	达标

全部 4 项核心分段质量指标达标。长度命中率在两份 DOCX 文档中略低于 100%（分别为 91.7% 和 93.3%），分析原因：DOCX 文档中存在超长表格（超过 1500 字符），按降级策略（`table_split_threshold=1500`）被拆分为多个子表格 chunk。拆分后的子表格 chunk 长度在 200-350 字符之间，低于 `min_chars=300`（部分子表格逻辑行较短），导致长度命中率下降。这些 chunk 在 `quality_flags` 中已标记为 `"oversized_table_split"`，属于预期的降级处理行为，不影响表格完整性保护。

产出 3 — `embedding.py` 降级机制确认

阅读并确认了 `backend/app/services/embedding.py`（124 行）的降级逻辑：

- 首选路径（第 48-65 行）：尝试导入 `sentence-transformers`，加载 `paraphrase-multilingual-MiniLM-L12-v2` 模型（118MB，384维）。模型下载到 `~/.cache/sentence-transformers/` 目录。
- 降级路径（第 68-85 行）：如果 `sentence-transformers` 导入失败或模型加载失败，使用 `fallback_encode()` 函

数。该函数将文本转为字符 3-gram 集合，使用 CRC32 哈希映射到 256 维向量空间。虽然是降级方案，但在小规模文本相似度计算中仍有一定区分度。

- 单例缓存（第 88-95 行）：`get_default_encoder()` 函数使用模块级 `_default_encoder` 变量存储单例，首次调用时初始化，后续调用直接返回。确保全局只加载一次模型。

当前开发环境中 `sentence-transformers` 已正常安装，MiniLM 模型工作正常。降级方案在 CI 环境或模型下载失败时作为安全网。

产出 4 — 模块整合建议

梳理了项目中存在功能重叠的两个模块：

1. `rag_store/`（ChromaDB 向量存储 + SQLite 元数据存储）：用于 RAG API 服务的持久化存储，支持文档上传后分段入库和检索查询。
2. `retrieval/`（EmbeddingStore + TF-IDFStore + SemanticStore）：用于评测和开发调试的内存检索，不依赖外部数据库。

两者的功能重叠点：都在做“chunk 存储 + 向量检索”。区别是 `rag_store` 面向生产（持久化、多文档、大规模），`retrieval` 面向开发（内存、快速、小规模）。

建议：后续可考虑统一检索接口，让 `EmbeddingStore` 和 `ChromaDB` 实现同一个检索抽象接口，使评测脚本可以无缝切换检索后端。这可以留到集成测试阶段（7月4日-6日）处理。

遇到的阻塞：

- 无重大阻塞。在性能分析过程中确认了 `embedding_store.py` 第 4 行的注释——“Uses pure numpy — no FAISS, no ChromaDB”——与项目实际情况一致。当前项目不依赖 FAISS，纯 `numpy` 方案在小规模评测中完全够用。后续如果评测规模增长（文档 >50 或 chunk >1000），可以考虑切换到 ChromaDB 的 HNSW 索引以获得更好的检索性能。

4.3 甘毅（内容组织负责人）

今日完成：

1. [T-8.7] 内容组织评测方案详细设计完成：确定了标签准确率、摘要忠实度、实体精度的评测方法、打分标准和数据格式 ✓
2. 启动了人工标注工作：从 3 篇文档中各抽取 5 个 chunk（共 15 个），已完成 `title.md` 的 4 个 chunk 标注 ✓
3. 深入分析了 `model_client.py` 中 `RuleBasedTagger` 的代码（约 200+ 行），理解了三个核心方法的实现细节和边界处理 ✓
4. 梳理了 `ContentOrganizer` 的双模式架构：LLM 优先（DeepSeek/OpenAI），失败静默降级到规则模式 ✓
5. 设计了标注数据格式和评测脚本的输入输出接口 ✓
6. 与孟之语对齐了评测方法论：标签准确率采用词级 Jaccard 重叠（阈值 0.5），摘要忠实度采用原文校验，实体精度采用精确匹配 + 类型校验 ✓

实际工时：约 6 小时

关键产出详情：

产出 1 — 内容组织评测方案 v1.0

一、评测对象

- 评测方法: `extract_tags()`、`generate_summary()`、`extract_entities()`
- 评测模式: 规则模式 (当前默认), LLM 模式后续对比评测

二、评测数据

- 来源: 3 份评测文档 (title.md、调研报告.docx、开题报告.docx)
- 抽样方法: 每文档随机抽取 5 个 chunk, 共 15 个 chunk
- 每个 chunk 的标注内容:
 - `gold_tags`: list[str] — 人工判定的标准关键词/标签 (5-8 个)
 - `gold_summary`: str — 人工撰写的标准摘要 (≤80 字, 仅包含原文信息)
 - `gold_entities`: list[dict] — 人工标注的标准实体列表
 - `{"type": "percentage", "value": "85%"}`
 - `{"type": "metric", "value": "Recall@5"}`
 - `{"type": "date", "value": "2026年6月"}`
 - `{"type": "org", "value": "XX大学"}`
 - `{"type": "version", "value": "v1.0"}`
 - `{"type": "threshold", "value": "≥90%"}`

三、评测指标定义

标签准确率 (Tag Accuracy) :

```

1  对于单个 chunk:
2    rule_tags = RuleBasedTagger.extract_tags(chunk.content) → ["tag1", "tag2", ...]
3    gold_tags = 人工标注的标准标签
4
5    对每个 rule_tag, 在所有 gold_tag 中找最佳匹配:
6      jaccard(rule_tag, gold_tag) = |jieba(rule_tag) ∩ jieba(gold_tag)| /
7      |jieba(rule_tag) ∪ jieba(gold_tag)|
8
9      如果存在 gold_tag 使得 jaccard ≥ 0.5, 则该 rule_tag 视为"命中"
10
11    chunk 级准确率 = 命中数 / len(rule_tags)
12
13  总体标签准确率 = 所有 chunk 的准确率的平均值

```

摘要忠实度 (Summary Faithfulness) :

```

1  对于单个 chunk:
2    rule_summary = RuleBasedTagger.generate_summary(chunk.content)
3    chunk_content = chunk 的原始文本
4
5    人工逐句检查 rule_summary 中每个陈述句:
6      - 如果该陈述能在 chunk_content 中找到对应的原文 → 忠实
7      - 如果该陈述引入了原文未出现的信息 → 不忠实
8
9    chunk 级忠实度 = 忠实的陈述句数 / 总陈述句数
10
11  总体摘要忠实度 = 所有 chunk 的忠实度的平均值

```

实体精度 (Entity Precision) :

```

1  对于单个 chunk:
2      rule_entities = RuleBasedTagger.extract_entities(chunk.content) → [{"type":
   "metric", "value": "Recall@5"}, ...]
3      gold_entities = 人工标注的标准实体列表
4
5      正确数 = rule_entities 中与 gold_entities 精确匹配 (类型 + 值完全一致) 的数量
6
7      chunk 级精确率 = 正确数 / len(rule_entities)
8      chunk 级召回率 = 正确数 / len(gold_entities)
9
10  总体实体精确率/召回率 = 所有 chunk 的微平均 (按实体总数计算)

```

四、验收标准

指标	目标	来源
标签准确率	≥ 85%	研发计划 7.2 指标表
摘要忠实度	≥ 90%	研发计划 7.2 指标表
实体识别精度 报告数值, 无硬性目标	—	—

产出 2 — RuleBasedTagger 代码分析

详细分析了 model_client.py 中 RuleBasedTagger 的三个核心方法:

extract_tags() 方法 (第 71-115 行左右)

处理流程:

1. 空文本检查 → 返回空列表
2. jieba 分词 (`_tokenise()`) → 词列表
3. 停用词过滤 → 去除约 55 个中英文停用词
4. TF-IDF 权重计算 (`_score_tokens()`): 使用 sklearn 的 TfidfVectorizer, 以 `document_corpus` (整个文档的文本) 作为背景语料计算 IDF, 使得在文档中过于常见的词被降权
5. bigram 组合优选: 将相邻的高分 unigram 合并为 bigram (如"语义" + "边界" → "语义边界")
6. 输出 top-k (默认 max_tags=5)

值得注意的设计:

- `document_corpus` 参数的引入是为了让 TF-IDF 的 IDF 部分有文档级的统计基础, 而非仅在单个 chunk 内计算。这提高了关键词的区分度——一个词在整个文档中频繁出现, 但在特定 chunk 中特别集中, 才是好的关键词。
- bigram 组合逻辑处理了中文分词的一个常见问题: jieba 分词倾向于将复合概念拆分为多个词 (如将"语义边界"拆为"语义"和"边界"), bigram 组合将高分 unigram 重新合并, 还原了复合概念。

generate_summary() 方法

处理流程:

1. 使用 splitter.py 中的 `split_sentences()` 将文本按句子切分
2. Lead-1: 取第一个非过渡句作为摘要首句 (过滤掉以"综上所述""总之""因此"等开头的过渡句)
3. TF-IDF 句子打分: 对剩余句子计算 TF-IDF 得分, 选取得分最高的 1-2 句作为补充
4. 拼接后截断至 ≤ 80 字

值得注意的设计:

- 摘要完全基于原文句子（抽取式），不生成新文本。这保证了忠实度天然为 100%（摘要中的所有文字都可以在原文中找到）。
- 过渡句过滤避免了摘要以无信息量的过渡句开头。

`extract_entities()` 方法

6 种正则模式（`ENTITY_PATTERNS`，第 25-37 行）：

模式	正则	示例
percentage	<code>\d+(?:\.\d+)?%</code>	85%、99.5%
threshold	<code>[>=<=<]+\s*\d+(?:\.\d+)?%</code>	≥90%、≤5%
metric	<code>(?:Recall Precision nDCG MRR F1 ROUGE BLEU Accuracy)(?:@\d+)?</code>	Recall@5、nDCG
date	<code>\d{4}\s*[年/-]\s*\d{1,2}\s*[月/-]\s*\d{1,2}\s*日?</code>	2026年6月22日
version	<code>[Vv]\d+(?:\.\d+)+</code>	v1.0、V2.3.1
org_suffix	<code>[-一隹]{2,16}(?:大学 学院 研究所 实验室 公司 部门 中心 委员会 课题组 团队 平台 系统 框架 模型)</code>	XX大学、XX实验室

正则模式的设计覆盖了研发计划中要求的实体类型。`org_suffix` 模式通过后缀词表（大学/学院/研究所/实验室/公司/部门/中心/委员会/课题组/团队/平台/系统/框架/模型）来识别机构名，这在技术文档和学术论文中是有效的策略——这些后缀词能覆盖绝大多数中文机构名的结尾。

产出 3 — `title.md` 初步标注结果

完成了 `title.md` 的 4 个 `chunk` 的关键词和摘要人工标注，作为标注方案的验证：

chunk	内容主题	人工标签（ <code>gold_tags</code> ）	人工摘要（ <code>gold_summary</code> ）
chunk_0001	课题背景与定位	课题11、RAG、智能分段、内容组织、智能体、专项能力、任务书	课题11面向RAG的智能分段与内容组织智能体，属于专项能力智能体，负责将结构化全文转换为适合RAG入库检索的chunk序列
chunk_0002	分段策略要求	结构感知、语义感知、不破句、表格保护、公式保护、代码保护、长度控制	分段策略要求按结构/语义感知方式分段，保证不破句，表格公式代码整体成块，chunk长度可控
chunk_0003	输出内容要求	摘要、关键词、主题标签、实体标签、标题路径、回链、元数据	每个chunk需生成文本、长度、标题路径、主题标签、关键词、摘要、实体标签和原文锚点回链
chunk_0004	评测与交付要求	HTTP API、baseline、Recall@k、nDCG、MRR、评测报告、接口文档、技术报告	提供HTTP API服务，构建固定长度baseline进行对比评测，交付评测报告、接口文档和技术报告

初步评测（将 `RuleBasedTagger` 的输出与人工标注对比）：

- 标签：4 个 `chunk` 的规则生成标签与人工标注的词级 Jaccard 重叠平均约 0.65（> 0.5 阈值），标签总体质量好。
- 摘要：抽取式摘要忠实度 100%（完全来自原文），信息覆盖度约 70-80%（抽取式摘要只覆盖了原文的部分关键信息）。
- 实体：5 类正则实体在 4 个 `chunk` 上均可正确识别，无漏检或误检。

正式评测需要完成全部 15 个 `chunk` 的标注后统一运行，明天（7月3日）完成。

遇到的阻塞：

- 人工标注耗时超出预期。每个 chunk 的完整标注（关键词+摘要+实体）平均需要 5-7 分钟，15 个 chunk 预计需要 1.5 小时。原因是学术文档（调研报告.docx）术语密集，关键词选择需要反复权衡，摘要撰写需要仔细核对原文。已调整时间预期，明天上午继续完成。
- 评价 RuleBasedTagger 的 extract_tags() 时，发现标签数量偏少（默认 max_tags=5），而部分内容密集的 chunk 可能需要 8-10 个标签才能充分描述。建议后续将 max_tags 作为可配置参数暴露给 ContentOrganizer。

4.4 王佳杭（评测与展示负责人）

今日完成：

1. [T-8.1] EmbeddingStore 向量索引构建：对 3 文档的 Smart 和 Fixed 分段结果分别构建了 numpy 向量索引（共 6 个索引）✓
2. [T-8.2] eval_rag.py 全量评测运行完成：计算了 Recall@1/3/5、Precision@5、nDCG@5、MRR，输出了 per-question 对比 ✓
3. [T-8.3] 与孟之语合作完成 Smart vs Fixed 第一版对比指标表，按文档和整体两个维度汇总 ✓
4. 对 15 个问题进行了 Smart vs Fixed 的逐题胜负统计，分析了胜负分布 ✓
5. 对每道 Smart 负于 Fixed 的题进行了简要的失败原因初步判断 ✓
6. 验证了 tune_segmenting_params.py 的可运行性：limit=10 的小规模搜索跑通，当前最优组合 Recall@5 差距缩窄至 -7.5% ✓
7. 整理了今日全部评测数据 ✓

实际工时：约 6 小时

关键产出详情：

产出 1 — 第一版 Smart vs Fixed 对比指标表

以下数据来自 eval_rag.py 的实际运行结果（与 README.md 记录一致）：

整体汇总（3文档×15题，宏平均）

指标	Smart（结构+语义）	Fixed-length（512字符）	差值	变化率
Recall@1	0.160	0.240	-0.080	-33.3%
Recall@3	0.455	0.407	+0.048	+11.8%
Recall@5	0.503	0.576	-0.073	-12.7%
Precision@5	0.307	0.320	-0.013	-4.1%
nDCG@5	0.556	0.700	-0.144	-20.6%
MRR	0.680	0.839	-0.159	-18.9%

按文档分类

文档	Smart R@1	Fixed R@1	Smart R@3	Fixed R@3	Smart R@5	Fixed R@5	Smart MRR	Fixed MRR
title.md	0.200	0.267	0.467	0.400	0.467	0.533	0.700	0.800
调研报告.docx	0.133	0.200	0.467	0.400	0.533	0.600	0.733	0.850
开题报告.docx	0.147	0.253	0.433	0.420	0.507	0.573	0.620	0.843

三个文档上的差距方向一致：Fixed 在 Recall@1（准确率）和 MRR（第一个相关结果排名）上领先，Smart 在 Recall@3（中等深度召回）上领先或持平。这印证了孟之语的根本分析——Smart 的较短 chunk 在 top-1 精准命中上处于劣势，但在 top-3 召回上结构感知的优势开始显现。

per-question 胜负统计 (15题)

结果	题数	占比	说明
Smart R@5 > Fixed R@5	5	33.3%	Smart 胜
Smart R@5 = Fixed R@5	3	20.0%	平局
Smart R@5 < Fixed R@5	7	46.7%	Fixed 胜

胜负比 5:7，差距仅 2 题。如果通过参数调优将 2 个负题转为胜题，整体 Recall@5 即可追平或反超。这给了我们信心——差距是可弥合的。

按问题类型分层（手动分类，样本量小，仅供参考）

问题类型	题数	Smart R@5	Fixed R@5	差值	胜负
定义类	4	0.533	0.500	+0.033	Smart 优
流程类	2	0.400	0.600	-0.200	Fixed 优
指标类	3	0.533	0.600	-0.067	Fixed 优
对比类	3	0.467	0.600	-0.133	Fixed 优
细节定位类	3	0.533	0.533	0.000	平局

Smart 在"定义类"问题上略优于 Fixed (+0.033)，验证了结构感知在精确定位概念定义时的优势。"流程类"差距最大 (-0.200)，但仅 2 题，样本太小，结论不可靠。

产出 2 — Embedding 检索流程确认

今天实际运行的检索流程：

```
1 1. 文档加载
2   eval_rag.py: load_and_segment()
3   |— doc_path → load_document() → list[DocumentBlock]
4   |— preprocess_document_blocks() → cleaned blocks
5   |— segment_blocks(cleaned, config) → smart_result["chunks"] (Smart 分段)
6   |— fixed_length_segment(raw_text) → baseline_chunks (Fixed 分段)
7
8 2. 向量索引构建
9   EmbeddingStore.add_chunks(doc_id, chunks)
10  |— _enrich_text(chunk) → "标题路径 + 正文" 增强文本
11  |— encoder.encode(texts) → MiniLM 384维向量 批量编码
12  |— _EmbeddingIndex(chunks, embeddings) numpy 矩阵存储
13
14 3. 检索查询
15   EmbeddingStore.search(doc_id, query, top_k=5)
16   |— encoder.encode([query]) → query_vec (384维)
17   |— np.dot(query_vec, chunk_embeddings.T) → scores (余弦相似度, 向量已L2归一化)
18   |— LexicalBoostRanker → 关键词微调 (最多+0.08)
19   |— 返回 top-5 hits: [{"chunk_id", "content", "score", ...}]
20
21 4. 相关性判定
22   EmbeddingRelevance.is_relevant(chunk_content)
23   |— keyword_score(chunk_content) >= 0.34? → True (关键词命中率 ≥ 1/3)
24   |— keyword score > 0 AND contains number or metric? → True (数字/指标宽松)
```

```

25 |     └─ embedding_score(chunk_content) >= threshold? → True (语义相似度 ≥ 0.45)
26 |
27 | 5. IR 指标计算
28 |     compute_ir_metrics(retrieved_chunks, relevance_judge, all_chunks)
29 |     └─ Recall@k = sum(relevant in top-k) / total_relevant_in_corpus
30 |     └─ Precision@k = sum(relevant in top-k) / k
31 |     └─ nDCG@k = DCG / IDCG (binary relevance: gain=1 if relevant, 0 otherwise)
32 |     └─ MRR = 1 / (rank of first relevant hit)

```

确认了 EmbeddingRelevance 的判定逻辑 (metrics.py 第 93-101 行) :

- 优先关键词匹配: `keyword_score >= 0.34` 直接判定相关 (不需要看 `embedding` 相似度)
- 数字/指标宽松: 含数字或指标证据的 `chunk`, 只要有关键词命中就判定相关
- 语义阈值: 以上两条都不满足时, 使用 `embedding` 余弦相似度判定 (默认阈值 0.45)
- 有关键词时, 语义阈值提高至 `max(threshold, 0.57)`, 防止纯语义相似产生假阳性

这个混合判定策略比纯 `embedding` 或纯关键词判定更鲁棒, 减少了因 `synonym mismatch` 导致的漏判。

产出 3 — `tune_segmenting_params.py` 验证

运行了 `limit=10` 的小规模验证测试。关键代码逻辑:

```

1 | tune_segmenting_params.py 核心流程:
2 | 1. SEARCH_SPACE 定义 6 维参数搜索空间
3 | 2. iter_configs() → 生成所有有效 SegmentConfig 组合 (过滤 max_chars < target_chars 等无效组合)
4 | 3. evaluate_config(config) → 对该配置运行一次完整评测, 返回 metrics dict
5 | 4. rank_key(config, metrics) → 按 improvement_pct → recall → precision → mrr 排序
6 | 5. 输出 top-N 最优配置

```

验证结果 (10 次采样中) :

- 搜索空间定义正确, 配置生成逻辑无 bug
- `evaluate_config()` 能正常完成单配置评测
- 当前最优组合 (10 次采样) : `target_chars=900, semantic_boundary_threshold=0.55, overlap_sentences=2` → `Recall@5` 提升至 0.533 (差距缩窄至 -7.5%)

重要发现: `threshold=0.55 + overlap=2` 的组合在现有搜索空间内已经能将差距从 -12.7% 缩窄到 -7.5%。这验证了孟之语根因分析的方向是正确的——提高语义阈值 (减少过度切分) 和增加 `overlap` 确实能改善 `Recall@5`。如果进一步增大 `target_chars` (从 900 到 1000-1200), 有机会进一步缩窄差距甚至转正。

遇到的阻塞:

- 评测数据集仅 15 题, `per-question` 波动较大。同一份文档的同一道题, 在不同参数配置下可能出现 `Recall@5` 跳变 0.2 以上的情况 (因为 `k=5` 的判断是离散的: 5 个 `chunk` 中任意一个命中/不命中都可能导致 `Recall@5` 变化 20%)。这使得小规模评测的灵敏度有限。明天扩充到 30 题后统计稳定性会有所改善。
- `eval_rag.py` 当前不支持命令行参数控制。要修改评测参数 (如 `SegmentConfig`), 需要直接编辑代码中的 `config = SegmentConfig()` 行。这不是大问题, 但降低了调试效率。建议后续增加 `--target-chars`、`-threshold` 等命令行参数。当前 `tune_segmenting_params.py` 已经通过 `evaluate_config()` 函数实现了参数化评测, 可以在对比实验日直接使用。

5 今日核心工作详解

5.1 分段引擎回顾与质量验证

今日在进入检索评测之前，先对分段引擎的核心质量进行了全面验证。以下基于实际代码分析分段引擎的完整工作流程：

分段主流程（`backend/app/services/segmenting/segmenter.py`，`segment_blocks()` 函数）：

```
1  输入: list[DocumentBlock] + SegmentConfig
2  输出: {"doc_id", "chunks", "statistics", "strategy"}
3
4  步骤1: build_candidate_chunks(blocks, config)
5      └─ splitter.py 第22行
6      └─ 遍历 blocks, 维护标题栈和当前 chunk 缓冲区
7      └─ 遇到新标题 → 如果当前缓冲区超过 heading_flush_min_chars, flush 当前 chunk
8      └─ 遇到表格/公式/代码 → 整体加入缓冲区 (不拆分), 标记 PROTECTED_BLOCK
9      └─ 语义边界检测 → 相邻段落 embedding 相似度 < threshold → flush 当前 chunk
10     └─ 超长文本块 → 按句子拆分
11
12  步骤2: split_oversized_chunks(candidates, config)
13     └─ splitter.py
14     └─ 对超过 max_chars 的 chunk 进行递归拆分
15     └─ 拆分点优先选择: 段落边界 > 句子边界 > 强制截断
16     └─ 表格/公式/代码块 → 如果超过 max_chars, 先尝试按逻辑行拆分, 标记
    "oversized_table_split"
17
18  步骤3: merge_short_chunks(normalized, config)
19     └─ splitter.py
20     └─ 对低于 min_chars 的 chunk 尝试与相邻 chunk 合并
21     └─ 合并条件: 同属一个父标题、非特殊块、合并后不超过 max_chars
22
23  步骤4: finalize_chunks(merged, doc_id, config)
24     └─ segmenter.py 第71行
25     └─ 生成稳定 chunk_id (格式: {doc_id}_chunk_{seq:04d})
26     └─ 构建 source_refs (block_id + page + metadata 回链)
27     └─ 设置 quality_flags (过大/过小/表格拆分/低置信等标记)
28     └─ 调用 enrichment 模块: 关键词提取、摘要生成、实体标注
```

质量统计（`backend/app/services/segmenting/statistics.py`）：

`build_statistics()` 函数返回的统计字典包含：

- `chunk_count`：chunk 总数
- `avg_chars`：平均字符数
- `target_length_hit_rate`：落在 `[min_chars, max_chars]` 区间的比例
- `source_ref_complete_rate`：包含 `source_refs` 的 chunk 比例
- `no_break_sentence_rate`：以合法句子边界结尾的 chunk 比例
- `table_code_formula_intact_rate`：特殊块未被标记 "split"/"truncated" 的比例
- `oversized_count` / `undersized_count`：过长/过短 chunk 数量

这些统计指标直接对应研发计划中的验收指标（不破句率、表格成块率、长度命中率、回链完整率）。

今日验证结果：3 份文档全部通过四项指标验收（详见 3.2 于贺汇报）。

5.2 评测框架架构分析

今日深入分析并运行了完整的评测框架。以下是架构层面的梳理：

评测框架的模块依赖图：

```
1 | scripts/eval_rag.py (228行)
2 | └─ backend/app/services/segmenting/
3 |   └─ segmenter.py (120+行) – segment_blocks() / segment_text()
4 |   └─ splitter.py (300+行) – build_candidate_chunks() / split / merge
5 |   └─ heading.py – 标题识别
6 |   └─ parser.py – 纯文本解析
7 |   └─ enrichment.py – 关键词/摘要/实体/回链
8 |   └─ statistics.py (133行) – 质量统计
9 |   └─ models.py – DocumentBlock, Chunk, SegmentConfig
10 | └─ backend/app/services/evaluation/
11 |   └─ baseline.py (93行) – fixed_length_segment()
12 |   └─ metrics.py (275行)
13 |     └─ EmbeddingRelevance – 相关性判定 (keyword + embedding 混合)
14 |     └─ compute_ir_metrics() – IR 指标计算
15 |     └─ run_segmenter_comparison() – 头对头对比 (eval_rag.py 未使用)
16 | └─ backend/app/services/retrieval/
17 |   └─ embedding_store.py (230行)
18 |     └─ EmbeddingStore – numpy 向量存储 + 余弦检索
19 |     └─ LexicalBoostRanker – 关键词微调
20 |     └─ _EmbeddingIndex – 内部索引数据类
21 | └─ backend/app/services/embedding.py (124行)
22 |   └─ EmbeddingEncoder – MiniLM 模型封装
23 |   └─ get_default_encoder() – 全局单例
24 |   └─ fallback_encode() – 字符 3-gram 降级
25 | └─ backend/app/services/document_loader/ – 多格式文档加载
26 | └─ backend/app/services/document_preprocessor.py – 预处理
27 | └─ backend/tests/eval_dataset.py (119行) – EVAL_DATASET 定义
```

评测框架的设计优点：

1. 模块解耦清晰：分段、检索、相关性判定、指标计算各自独立，可以替换任意一个环节而不影响其他环节。例如，可以将检索后端从 `EmbeddingStore` 替换为 `ChromaDB`，只需修改 `eval_rag.py` 中的 `store` 实例化代码。
2. 相关性判定鲁棒：`EmbeddingRelevance` 的混合判定策略 (`keyword + embedding + metric` 宽松) 考虑了多种匹配情况，减少了因 `synonym mismatch`、缩写/全称不匹配、数值近似匹配等导致的漏判。
3. 评测数据外部化：`EVAL_DATASET` 定义在独立的 `eval_dataset.py` 中，评测逻辑在 `eval_rag.py` 中。修改评测数据不需要修改评测逻辑，反之亦然。

评测框架的已知局限：

1. `Fixed-length baseline` 的检索也使用了同一 `EmbeddingStore` 和 `EmbeddingRelevance`，确保了两者的对比是公平的（相同的检索后端 + 相同的相关性判定标准）。
2. 评测使用 `EmbeddingStore`（内存检索）而非 `ChromaDB`（持久化检索）。`EmbeddingStore` 在每次评测运行时重新构建索引，这确保了每次评测的索引都是一致的新构建，避免了缓存污染。但如果后续需要评测大规模数据集（100+ 文档），内存索引可能不够用。
3. `eval_rag.py` 当前没有命令行参数接口，修改评测参数需要编辑代码。`tune_segmenting_params.py` 通过 `evaluate_config()` 实现了参数化，但 `eval_rag.py` 本身没有跟进。

5.3 Embedding 检索流程优化

今日对于贺负责的 Embedding 检索流程进行了性能分析和轻量优化。

已有优化机制确认：

1. 全局单例缓存（`embedding.py` 第 88-95 行）：`get_default_encoder()` 返回模块级单例 `_default_encoder`，确保整个评测过程中 MiniLM 模型只加载一次。首次加载耗时约 15 秒，后续调用耗时 < 1ms。
2. 批量编码（`EmbeddingStore.add_chunks()` 第 58-59 行）：所有 chunk 的 `enriched_text` 一次性传入 `encoder.encode()`，由 sentence-transformers 内部进行批量推理。相比逐条编码，批量编码效率提升约 3-4 倍。
3. L2 归一化预计算（`EmbeddingIndex._init_()`）：chunk embedding 在索引构建时进行 L2 归一化，检索时直接使用归一化后的向量计算内积（等价于余弦相似度），避免了每次检索都重新归一化。
4. 降级安全网（`embedding.py` 第 68-85 行）：当 sentence-transformers 不可用时，自动降级到字符 3-gram 哈希向量。虽然精度降低，但确保了评测脚本在任何环境下都能运行（不会因为缺少依赖而崩溃）。

轻量优化：

- 确认了 `enrich_text()` 函数（`embedding_store.py` 第 196-216 行）的内容：标题路径（`title_path` 用 ">" 连接）+ "\n" + 正文。标题路径信息被拼接在正文之前，在 MiniLM 的 self-attention 机制中能够与正文产生交互。但由于标题路径通常很短（20-50 字符），在 384 维的 embedding 向量中贡献有限。这为后续优化提供了方向（如：将标题路径重复 3 次以增加其权重）。
- 确认了 `LexicalBoostRanker` 的微调逻辑：对包含 query 中关键词的 chunk，在 cosine similarity 基础上增加最多 0.08 的微调分数。这个微调幅度很小（cosine similarity 通常在 0.2-0.8 之间，0.08 的微调相当于 10-40% 的相对提升），不会改变排序的主要格局，但能在关键词明确的情况下给匹配的 chunk 一个轻微的提升。

5.4 Smart vs Fixed 评测运行与结果分析

今日的核心工作是对 Smart 和 Fixed 两种分段策略进行全量检索评测。评测的运行和结果已在 3.4 王佳杭汇报中详细列出。这里对结果进行更深层的分析。

Smart 什么时候赢？

分析 5 个 Smart 胜出的问题，发现它们有以下共同特征：

- 问题的答案集中在文档的一个明确的结构单元（一个标题下的完整段落），不跨标题
- 答案的关键词与 chunk 的标题路径高度相关（如“分段策略中需要保护哪些特殊内容块？”→ 答案在“分段策略要求”标题下的段落）
- 问题的 `answer_keywords` 集中在单一主题上，不分散

在这些场景下，Smart 的标题边界优先策略能够生成边界精准的 chunk，chunk 内部的语义连贯性强，embedding 向量与该主题的匹配度高。

Smart 什么时候输？

分析 7 个 Smart 败出的问题，发现它们有以下共同特征：

- 问题的答案分布在文档的多个位置（如“课题要求交付哪些东西？”→ 答案分散在多个段落）
- 问题的表述较宽泛，没有明确的标题层级线索（如“报告中提到了哪些技术难点？”）
- 问题的 `answer_keywords` 涉及多个子主题，需要跨 chunk 信息综合

在这些场景下，Fixed 长 chunk 的信息密度优势显现。一个 512 字符的 Fixed chunk 可能覆盖了 Smart 两个 400 字符

核心洞察：

Smart 和 Fixed 的优劣取决于问题的"聚焦度"：

- 聚焦问题（答案在单一结构单元内）→ Smart 优
- 宽泛问题（答案跨多个结构单元）→ Fixed 优

这个洞察为后续的参数调优提供了方向：不是要 Smart 在所有问题上都优于 Fixed（这不可能，因为两者有各自的适用场景），而是要找到一个参数配置，使 Smart 在聚焦问题上的优势 > 在宽泛问题上的劣势。提高 target_chars（让每个 Smart chunk 更长）可以在保持结构感知优势的同时，缩小信息密度的差距。

5.5 内容组织模块评测准备

今日甘毅为内容组织质量评测做了详细的准备工作。以下是评测准备的完整梳理：

评测对象（代码层面）：

`backend/app/services/organizer/model_client.py` 中的 `RuleBasedTagger` 类：

- `extract_tags(text, document_corpus)` → list[str]: jieba 分词 + TF-IDF + bigram 组合 → 关键词标签
- `generate_summary(text)` → str: Lead-1 + TF-IDF 句子打分 → 抽取式摘要 (≤80字)
- `extract_entities(text)` → list[dict]: 6 种正则模式匹配 → 实体列表

`backend/app/services/organizer/organizer.py` 中的 `ContentOrganizer` 类：

- `organize_chunk(content, document_context)` → OrganizeResult: 编排上述三个方法，LLM 优先、规则降级
- `organize_batch(chunks)` → list[OrganizeResult]: 批量处理

评测方法（与 6月26日 P0 方法论文档一致）：

标签准确率 → 词级 Jaccard 重叠 (jieba 分词后计算交集/并集，阈值 0.5)

摘要忠实度 → 人工判定 (摘要中每个陈述是否能在原文中找到)

实体精度 → 精确匹配 (类型 + 值完全一致)

标注数据：

15 个 chunk (3文档×5chunk) 的人工标注正在进行中。每个 chunk 标注三样东西：gold_tags (标准标签列表)、gold_summary (标准摘要)、gold_entities (标准实体列表)。标注完成后，将 RuleBasedTagger 的输出与 gold 标注对比，计算三个质量指标。

当前进展：title.md (4 chunk) 已完成初步标注和对比。调研报告.docx (5 chunk) 和开题报告.docx (5 chunk) 的标注正在进行中，预计明天上午完成。正式评测运行安排在明天 (7月3日)。

5.6 参数调优框架验证

今日王佳杭对 `scripts/tune_segmenting_params.py` 进行了验证性测试，确认了搜索框架可以正常用于明天的对比实验日。

搜索空间定义 (tune_segmenting_params.py 第 12-20 行)：

```

1 SEARCH_SPACE = {
2     "min_chars": [220, 260, 300],
3     "target_chars": [650, 750, 900],
4     "max_chars": [900, 1000, 1200],
5     "heading_flush_min_chars": [120, 180, 240, 300],
6     "semantic_boundary_threshold": [0.35, 0.45, 0.55],
7     "overlap_sentences": [0, 1, 2],
8 }

```

搜索链路:

```

1 SEARCH_SPACE → iter_configs() → 生成所有有效 SegmentConfig 组合
2   → evaluate_config(config) → 单配置评测
3   → segment_blocks(blocks, config) → Smart chunks
4   → fixed_length_segment(text) → Baseline chunks
5   → EmbeddingStore + 检索 + EmbeddingRelevance
6   → compute_ir_metrics → 返回 metrics
7   → rank_key(config, metrics) → 排序
8   → 输出 top-N 最优配置

```

验证结果:

- limit=10 小规模搜索跑通, 确认了框架的完整性和正确性
- 当前最优组合 (10 次采样): target_chars=900, threshold=0.55, overlap=2 → Recall@5=0.533 (-7.5% vs Fixed)
- 验证了孟之语的根因假设: 提高 threshold (0.45→0.55) 和增加 overlap (1→2) 确实能改善 Recall@5

明天需要做的调整:

- target_chars 候选值增加 1000、1100、1200 三个值
- 调整后的搜索空间: $6 \times 3 \times 3 \times 4 \times 6 \times 5 = 6480$ 种组合 (过滤无效组合后约 3000+)
- 由于评测数据集只有 15 题 (明天扩充到 30 题), 单次评测耗时约 20-30 秒, 全部搜索不现实
- 采用两阶段搜索策略: 第一阶段搜索 target_chars + min_chars/max_chars (18 种), 找出最优长度配置; 第二阶段在最优长度附近搜索 threshold + overlap + heading_flush (约 50-100 种), 精调语义和上下文参数

6 问题与解决方案

6.1 今日新发现的问题

编号	问题描述	严重程度	发现人	解决方案	状态
P-7.2-1	Smart Recall@5 低于 Fixed 12.7%: chunk 长度偏短是主因, 语义阈值偏敏感是次因	高	孟之语、王佳杭	target_chars 900→1000-1100, threshold 0.45→0.50-0.55, overlap 1→2-3	方案已定, 7月3日执行
P-7.2-2	评测数据集仅 3 文档×15题=15题, 规模偏小, per-question 波动对平均指标影响大	中	孟之语	7月3日上午扩充至每文档10题 (30 题), 后续视情况继续扩充	待执行
P-7.2-3	heading_based baseline 尚未实现, 三策略对比缺一个维度	中	孟之语	7月3日评估工作量, 如时间允许当天实现 (预计2-3h)	待执行
P-7.2-4	eval_rag.py 缺少命令行参数接口, 修改评测配置需要编辑代码	低	王佳杭	不影响功能, 后续集成测试阶段改进	已记录
P-7.2-5	人工标注耗时超出预期 (每chunk 5-7分钟), 15 chunk 需 1.5h	低	甘毅	调整预期时间, 明天上午继续完成	已调整

编号	问题描述	严重程度	发现人	解决方案	状态
P-7.2-6	retrieval_text 中标题路径权重不足 (<10%)，结构信息在 embedding 中被稀释	低	于贺	后续可考虑标题路径重复拼接增加权重	已记录
P-7.2-7	rag_store/ 和 retrieval/ 存在功能重叠（均为"chunk存储+检索"）	低	于贺	后续可统一检索接口，集成测试阶段处理	已记录

6.2 遗留问题追踪

编号	问题描述	发现日期	责任人	当前状态	预计解决
P-6.28-1	embedding 模型下载可能失败	6月28日	孟之语	降级方案（字符3-gram）已就绪，当前 MiniLM正常工作	已降级处理
P-6.29-1	LLM API 不可用时摘要和标签降级	6月29日	甘毅	ContentOrganizer 的 LLM→规则降级链路已实现，验证通过	已解决
P-7.1-1	Smart vs Baseline Recall@5 差距 -12.7%	7月1日	孟之语、王佳杭	今日根因已定位（3要素），调优方案已明确	7月3日执行

7 晚间同步会记录（21:30–21:55）

7.1 各成员今日完成总结

孟之语汇报：

今天主要精力放在根因分析上。通过分析 chunk 长度分布、语义边界触发逻辑和 per-question 评测结果，确定了三个根因：chunk 平均长度偏短（主因）、语义阈值偏敏感（次因）、overlap 不足 + 标题权重偏低（辅因）。

根因分析的最大收获是明确了参数调优方向。目前的 tune_segmenting_params.py 搜索空间中，target_chars 最大只有 900。明天需要增加 1000、1100、1200 三个候选值，扩大搜索范围。王佳杭的验证测试已经证明，仅调整 threshold（0.45→0.55）和 overlap（1→2）就能将差距从 -12.7% 缩窄到 -7.5%。如果再加上 target_chars 的调整，有机会进一步缩窄甚至转正。

另外，今天整理了技术报告的大纲（7 章 28 节），基本覆盖了研发计划要求的所有交付物内容。后续各章节的详细内容需要各位成员根据自己的负责模块来填充。

今日工时约 6.5h。

于贺汇报：

今天完成了三件事。一是 Embedding 检索流程的完整性能分析，每个环节的耗时都测了一遍。整体一次评测约 19 秒，可以接受。主要瓶颈在 embedding 编码和相关性判定两处，后续如果需要加速，改批量编码即可。

二是分段质量统计验证，三份文档的四项指标全部达标。这是第二次验证了（上次是 6月30日在 M3 验收时做的），两次验证结果一致，数据可信。

三是发现了一些可以整合优化的点，比如 rag_store/ 和 retrieval/ 的功能重叠、retrieval_text 中标题权重不足的问题。这些不紧急，已记录，集成测试阶段再处理。

今日工时约 5.5h。

甘毅汇报：

今天主要做了内容组织评测的方案设计和标注启动。评测方案写得很详细，包含了三个指标的完整定义、计算方法和验收标准。标注方面，title.md 的 4 个 chunk 已经完成，初步对比结果还可以——标签 Jaccard 重叠约 0.65，摘要忠实度 100%（因为是抽取式的，天然忠实）。剩余 11 个 chunk 的标注明天上午继续。

另外，深入分析了一下 RuleBasedTagger 的代码。发现 extract_tags 的默认 max_tags=5 对于内容密集的 chunk 可能偏少。后续可以考虑做成可配置参数。extract_entities 的 6 种正则覆盖了研发计划要求的实体类型，正则写得比较严谨。

今日工时约 6h。

王佳杭汇报：

今天主要做了三件事。一是全量评测，产出了 Smart vs Fixed 的完整对比指标表。结果和 README 中记录的一致：Smart Recall@5=0.503、Fixed=0.576，差距-12.7%。但逐题看，15 题中 Smart 胜 5、平 3、负 7，差距只有 2 题，可弥合。

二是 per-question 分析。Smart 在定义类问题上表现好（+0.033），在流程类和对比类问题上表现差。这和孟之语的根因分析结论一致——定义类问题答案集中在一个结构单元内，Smart 的结构感知有优势；对比类问题答案需要跨 chunk 综合，Smart 的短 chunk 劣势显现。

三是 tune 脚本的验证。limit=10 跑通了，当前最优组合已经能把差距缩窄到 -7.5%。明天的搜索方向明确：扩大 target_chars 搜索范围，两阶段搜索策略。

今日工时约 6h。

7.2 全组讨论

讨论 1 — 参数调优的成功概率

王佳杭：验证测试中，仅调整 threshold + overlap 就已经改善了 5.2 个百分点（-12.7% → -7.5%）。如果再加上 target_chars 的上调（900→1100），预计可以再改善 5-8 个百分点。综合来看，有 70-80% 的概率能将差距缩窄到 -5% 以内，有 40-50% 的概率能转正（达到 +10% 目标的概率约 20-30%）。

孟之语：即使最终不能达到 +10%，只要差距明显缩窄（比如到 -3% 以内），我们就能在技术报告中给出合理的解释——Smart 策略在结构感知上的优势体现在 Recall@3（+11.8%）和定义类问题的精准定位上，Recall@5 在 15 题小样本上的差距可以通过增加 target_chars 来改善。如果数据集更大、问题更多样，差距可能进一步缩小。

于贺：同意。+10% 是一个比较激进的目标。Smart vs Fixed 在 Recall@3 上已经 +11.8%，这说明在“深度适中的检索”场景下，Smart 是明确优于 Fixed 的。Recall@5 的差距可以通过参数调优改善。nDCG 和 MRR 的差距与 Recall@1 高度相关（因为 top-1 命中率决定排序质量），Recall@1 的改善会带动 nDCG 和 MRR 同步改善。

讨论 2 — heading_based baseline 的实现计划

孟之语：heading_based baseline 的核心逻辑是：仅利用 heading.py 的标题识别 + splitter.py 的长度控制，不启用 semantic_boundary（设置 enable_semantic_boundary=False 或将 threshold 设得极高）。实现起来其实是复用现有的分段框架，只是关闭了语义边界功能。工作量估计 2-3 小时。

王佳杭：那明天的评测就从双策略扩展到三策略。先跑 Smart + Fixed + Heading 的三策略全量对比，确认 Heading 在各个指标上的位置（预期介于 Smart 和 Fixed 之间，或者和 Smart 接近），然后重点对 Smart 做参数调优。

甘毅：Heading-based 和 Smart 的对比很有价值——两者都有结构感知（标题边界），区别仅在于是否有语义边界增强。这个对比可以量化“语义信息的增量价值”。如果 Heading 和 Smart 的差距很小，说明语义边界的贡献有限；如果差距明显，说明语义边界确实有独立于结构信息的价值。

讨论 3 — 与老师沟通的时机和内容

孟之语：按照研发计划 9.4 节，"关键里程碑延迟超过 1 天"需要与老师沟通。目前 M4 本质上还没有延迟——评测框架已经能输出三个 IR 指标，满足 M4 的基本要求。但 Recall@5 指标未达标。明天对比实验日结束后，不管结果如何，都向老师汇报一次当前进展。

汇报内容建议包含四点（按研发计划要求的格式）：

1. 当前完成情况：M1-M3 已通过，M4 评测框架已运行，Smart vs Fixed 对比完成，分段质量和内容组织评测进行中
2. 偏差说明：Recall@5 当前 -12.7%，根因已定位（chunk 长度偏短），正在通过参数调优改善
3. 对最终交付的影响：核心功能不受影响（分段、内容组织、API 均已实现），评测指标可能不能达到 +10% 目标，但可以在报告中详细分析原因
4. 调整方案：明天完成参数调优 + heading_based baseline + 内容组织评测，7月4日转入集成测试

讨论 4 — 后续日程的紧凑程度

孟之语：距离 7月10日最终提交还有 8 天。接下来几天的工作密度会比较大：

- 7月3日：对比实验日（参数调优 + heading_based + 内容组织评测）
- 7月4日：集成修复日（FastAPI 联调、命令行脚本、日志、配置）
- 7月5日：测试日（单元测试、接口测试、样例回归、人工抽样）
- 7月6日：演示日（演示页面/HTML报告）
- 7月7-9日：文档+报告+冻结

提醒大家，7月3日是最后一个可以进行大方向调整的日子。7月4日起，原则上不新增功能，只做修复、测试和文档。如果需要做任何大改动，必须在7月3日完成。

8 组长总结

8.1 今日整体评价

今天是阶段4（评测阶段）的第2天，也是整个项目周期的第11天。全组按计划完成了检索评测日的全部 8 项任务。

好消息：

1. 第一版 Smart vs Fixed 对比指标表正式确认，3 个 IR 指标（Recall@k、nDCG@k、MRR）均可稳定输出。评测框架工作可靠，数据可复现。
2. 分段质量 4 项统计指标在 3 份文档上全部达标（不破句率 100%、表格成块率 100%、长度命中率 94.4%、回链完整率 100%），证明了分段引擎的核心质量。
3. Recall@5 差距 -12.7% 的根因已明确（chunk 长度偏短 + 语义阈值偏敏感 + overlap 不足），参数调优方向清晰。已有的 tune_segmenting_params.py 验证测试证明，仅两个参数（threshold + overlap）的调整就能将差距从 -12.7% 缩窄到 -7.5%，说明方向正确。
4. Smart 在 Recall@3 上已经领先 Fixed +11.8%，在定义类问题上领先 +0.033。这证明了结构感知分段的价值——当查询需要精确定位时，Smart 的标题边界优先策略是有效的。
5. 内容组织评测方案已详细设计，与 6 月 26 日 P0 方法论修复保持一致，评测方法学上有连贯性。人工标注已启动，明天可产出初步评测结果。
6. 评测基础设施状态良好：EmbeddingStore 工作稳定、EmbeddingRelevance 判定鲁棒、tune 搜索框架验证可用、所有脚本均可正常运行。

挑战：

- 1. Recall@5 差距仍需弥合。明天是对比实验日，是参数调优的关键窗口。需要在一天内完成搜索空间扩大、网格搜索、最优参数确定、全量复测、badcase 分析和对比报告初版。
- 2. heading_based baseline 尚未实现。明天需要在参数调优的同时完成实现和三策略对比，时间紧张但可行。
- 3. 评测数据集规模偏小（15 题），明天上午需要扩充到 30 题。甘毅和王佳杭需要协调完成。
- 4. 距离最终交付还有 8 天，文档、演示材料等产出物尚未开始系统整理。需要从 7月4日起集中精力处理。

8.2 项目进度对照

按照研发计划，今天是第 11 天。对照阶段 4 计划：

计划内容	计划状态	实际状态	偏差分析
7月1日 评测准备日	完成 baseline + 准备问题集	fixed_length baseline 已有；问题集 15 题已梳理；heading_based 未实现	heading_based 缺失是唯一偏差
7月2日 检索评测日	建立向量索引 + 计算指标 + 形成指标表	全部完成：EmbeddingStore 索引 + IR 指标 + 对比指标表 + 根因分析	按计划完成，无偏差
7月3日 对比实验日	多策略评测 + badcase + 确定默认参数	待进行	按计划推进

对最终交付的影响评估：

- 当前进度基本符合计划节奏，未出现关键路径延迟
- heading_based 缺失可在 7 月 3 日补上，不影响整体进度
- 最大风险仍是 Recall@5 达标问题，但根因清晰、方案明确
- 如果 7 月 3 日参数调优后仍不能达标，启动降级方案：在技术报告中详细说明原因、已尝试的调优路径、Smart 在 Recall@3 的显著优势、以及后续改进方向

8.3 组员今日表现

成员	任务完成	工时	突出贡献
孟之语	2+2 项	6.5h	Recall@5 根因分析 3 要素定位，三阶段调优方案，技术报告大纲
于贺	2+1 项	5.5h	评测性能全链路分析，分段质量 3 文档验证通过，模块整合建议
甘毅	2 项	6h	内容组织评测方案 v1.0, RuleBasedTagger 深入代码分析，标注启动
王佳杭	3+2 项	6h	全量评测运行，per-question 分析，tune 框架验证，最优组合发现

全组今日计划完成率 100%。每位成员完成了分配的所有任务，工作质量良好，产出物可复用。

8.4 需向上沟通的事项

- 1. Recall@5 差距：当前 -12.7%。明天参数调优后预计可改善至 -5% 以内甚至转正。如果改善后的差距仍不能达到 +10% 目标，计划在 7 月 3 日晚间向老师汇报当前进展和调整方案。
- 2. heading_based baseline 缺失：当前仅有 fixed_length baseline。计划 7 月 3 日补充实现 heading_based baseline（按标题切分+长度控制，不启用语义边界），使评测达到研发计划要求的三策略对比（fixed_length / heading_based / structure_semantic_hybrid）。
- 3. 评测数据集规模：当前 15 题偏少。计划明天扩充到 30 题。如果需要更大规模（50+ 题），需要评估标注人力和时间。

9 关键决策记录

编号	日期	决策内容	决策人	依据
D-7.2-1	7月2日	今日评测以 Smart vs Fixed 双策略为主，heading_based 推迟到 7月3日实现	孟之语	heading_based 尚未实现，今天集中精力做根因分析
D-7.2-2	7月2日	标签准确率评测采用词级 Jaccard 重叠 (jieba 分词 + 阈值 0.5)	甘毅、孟之语	与 6月26日 P0 方法论修复保持一致；规则标签与人工标注表述不同，精确匹配会低估
D-7.2-3	7月2日	评测数据集暂用现有 15 题出第一版指标表，明天扩充至 30 题后再做参数搜索	全组	平衡评测效率和统计稳定性
D-7.2-4	7月2日	参数调优利用已有 tune_segmenting_params.py，扩大 target_chars 候选值 (增加 1000-1200)	孟之语、王佳杭	避免重复造轮子；验证测试已证明框架可用
D-7.2-5	7月2日	M4 里程碑正式验收推迟到 7月3日对比实验完成后	全组	heading_based baseline 和全参数调优未完成，一并验收
D-7.2-6	7月2日	如 7月3日参数调优后 Recall@5 差距仍 >10%，当晚向老师汇报	孟之语	研发计划 9.4 节规定：关键里程碑偏差需及时沟通
D-7.2-7	7月2日	采用两阶段参数搜索策略：粗调长度参数 (18种) → 细调语义+重叠参数 (~50-100种)	王佳杭、孟之语	控制搜索时间在 1-1.5 小时内完成
D-7.2-8	7月2日	eval_rag.py 的 JSON 输出格式改进推迟到集成测试阶段	于贺、孟之语	当前手动整理可满足需求，后续统一改进

10 明日计划（7月3日 对比实验日）

10.1 明日定位

根据研发计划 4.2 节，7月3日为**对比实验日**：

对比实验日：对多策略跑评测，分析失败案例，确定最终默认策略参数

负责人：王佳杭、孟之语、甘毅

产出：对比报告初版

里程碑：M4 — 能输出对比实验结果 (baseline 与智能策略完成对比，输出 Recall@k、nDCG、MRR)

10.2 明日任务分配

编号	任务	负责人	预计工时	优先级	依赖
T-9.1	评测数据集扩充：每文档新增 5 题（扩至 3 文档×10 题=30 题），覆盖 5 种问题类型	甘毅、王佳杭	2h	P1	—
T-9.2	实现 heading_based baseline：复用 heading.py + splitter.py，关闭语义边界	孟之语	2.5h	P1	—
T-9.3	参数网格搜索（两阶段）：扩大 target_chars 候选值 + 全量搜索	王佳杭	2h	P0	T-9.1
T-9.4	分析网格搜索结果，确定最优默认策略参数	孟之语、王佳杭	1.5h	P0	T-9.3
T-9.5	最优参数下三策略全量评测：Smart vs Fixed vs Heading-based	王佳杭	1h	P0	T-9.2, T-9.4
T-9.6	深度分析 badcase（≥5 个），按失败模式分类，撰写改进建议	孟之语	2h	P0	T-9.5
T-9.7	完成全部 15 chunk 人工标注 + 运行内容组织质量评测	甘毅	2.5h	P1	—
T-9.8	撰写对比报告初版：指标表 + 分层分析 + badcase + 策略推荐	孟之语、王佳杭	2h	P1	T-9.5, T-9.6
T-9.9	验证 M4 全部验收标准，记录验收结果	于贺	1h	P1	T-9.5, T-9.7

10.3 参数搜索详细计划

搜索策略：两阶段搜索

第一阶段 — 粗调长度参数：

- 参数：target_chars × min_chars/max_chars
- target_chars 候选值：[650, 750, 900, 1000, 1100, 1200]（新增 1000-1200）
- min_chars 候选值：[220, 260, 300, 400]（新增 400）
- max_chars 候选值：[900, 1000, 1200, 1400]（新增 1400）
- 其他参数固定：threshold=0.50, overlap=2, heading_flush=240
- 有效组合数：约 20-30 种
- 预计耗时：约 10-15 分钟

第二阶段 — 细调语义 + 上下文参数：

- 在第一阶段最优长度配置附近搜索
- 参数：semantic_boundary_threshold × overlap_sentences × heading_flush_min_chars
- threshold 候选值：[0.40, 0.45, 0.50, 0.55, 0.60]
- overlap 候选值：[0, 1, 2, 3]
- heading_flush 候选值：[180, 240, 300]
- 有效组合数：约 60 种
- 预计耗时：约 30-40 分钟

总计约 80-100 种配置组合，预计总耗时 45-60 分钟（基于单次评测 ~30 秒估计）。如果数据集扩充到 30 题，单次评测可能延长到 ~40 秒，总耗时约 60-75 分钟。

10.4 明日目标

1. 主要目标: 通过参数调优将 Recall@5 差距缩窄至 -5% 以内或转正, 确定默认策略参数, 完成三策略对比
2. 次要目标: 完成 badcase 深度分析 (≥5个), 完成内容组织质量初步评测, 产出对比报告初版
3. 验收目标: M4 里程碑正式验收通过 (baseline 与智能策略完成对比, Recall@k、nDCG、MRR 可输出)

10.5 明日风险提示

1. 如果参数调优后 Recall@5 仍未达标 (< +10% 且差距 > 5%), 需要在当天晚间向老师汇报情况, 说明根因和调整方案。
2. 网格搜索如果耗时过长 (> 1.5 小时), 优先完成第一阶段 (长度粗调) 并选取最优候选, 第二阶段可以缩减搜索范围或推迟。
3. 明天是周五, 如果组员有出行计划, 务必保证上午完成核心实验 (参数搜索至少完成第一阶段), 下午做分析和报告撰写。

11 项目整体进度追踪

11.1 项目日历 (19 天)

1	阶段0: 6/22 [启动日]		✓ 已完成
2	阶段1: 6/23-6/24 [架构设计]	M1: 6/24	✓ 已完成
3	阶段2: 6/25-6/27 [基础分段]	M2: 6/27	✓ 已完成
4	阶段3: 6/28-6/30 [语义+内容]	M3: 6/30	✓ 已完成
5	阶段4: 7/01-7/03 [评测]	M4: 7/3	← 当前 (约 60%)
6	阶段5: 7/04-7/06 [集成测试]	M5: 7/6	待进行
7	阶段6: 7/07-7/09 [文档+答辩]	M6: 7/9	待进行
8	阶段7: 7/10 [最终提交]	M7: 7/10	待进行

11.2 已完成里程碑

里程碑	日期	验收标准	验收结果	状态
M1	6月24日	schema、API、样例数据确定, 能说明输入输出	通过	已完成
M2	6月27日	/segment 可运行, 能输出基础 chunk、title_path、source_refs	通过	已完成
M3	6月30日	语义增强、特殊块保护、摘要和标签初步可用	10/10 通过	已完成

11.3 进行中里程碑

里程碑	目标日期	验收标准	当前进度
M4	7月3日	baseline 与智能策略完成对比, 能输出 Recall@k、nDCG、MRR	约 75% (fixed_length 对比完成, 指标可输出; heading_based 待实现; 参数调优待完成; Recall@5 待改善)

11.4 待完成里程碑

里程碑	目标日期	验收标准
M5	7月6日	服务、评测、演示材料完成候选版本
M6	7月9日	代码、报告、数据、演示全部冻结
M7	7月10日	最终提交和验收展示

11.5 工时统计

阶段	计划工时（4人合计）	实际工时（累计估算）	偏差
阶段0（1天）	17h	~15h	-12%
阶段1（2天）	25h	~22h	-12%
阶段2（3天）	40h	~38h	-5%
阶段3（3天）	97h	~100h	+3%
阶段4（至7月2日，2天）	~28h（占阶段4 42h的67%）	~47h（含7月1-2日）	按比例正常
累计（11天）	~207h	~222h	+7%

工时略超计划 7%（+15h），主要原因是阶段3 语义边界调试和内容组织评测标注超出预期。仍在计划缓冲范围内（总缓冲 43h，已用约 15h，剩余 28h）。

12 任务状态表

按照研发计划 2.2 节 15 个研发步骤进行状态追踪：

步骤	研发内容	状态	完成度	负责人	关键产出
1	任务书解读与需求边界确定	已完成	100%	孟之语	需求清单、功能优先级、验收指标
2	输入输出结构设计	已完成	100%	于贺	DocumentNode、Chunk、SegmentConfig、EvaluationResult
3	API 接口设计	已完成	100%	于贺	/health、/segment、/organize、/evaluate、/strategies
4	样例数据准备	已完成	100%	于贺、甘毅	3 份结构化文档（title.md + 2 DOCX）
5	输入适配与校验	已完成	100%	于贺	schema 校验、节点标准化、异常 warnings
6	基础分段算法	已完成	100%	孟之语	标题候选块、长度统计、过长拆分、过短合并
7	结构/语义融合分段	已完成	100%	孟之语	语义相似度、边界评分、标题权重、段落连续性
8	特殊块保护	已完成	100%	孟之语、甘毅	表格/公式/代码/列表保护、超长表格降级处理
9	overlap 与上下文控制	已完成	100%	孟之语	可配置重叠、上下文补充
10	内容组织	已完成	100%	甘毅	摘要、关键词、主题标签、实体标签、管理标签
11	回链与元数据	已完成	100%	于贺	title_path、source_refs、strategy_info、quality_flags
12	基线与检索评测	进行中	70%	王佳杭、孟之语	fixed_length 对比完成，heading_based 待实现，指标表已产出
13	服务集成	待进行	0%	于贺、孟之语	7月4日开始
14	测试与质量检查	部分完成	40%	全员	单元测试 18 个通过，集成测试 7月5日
15	演示与文档	待进行	10%	王佳杭、甘毅	素材积累中，正式撰写 7月7-9日

12.1 验收指标对照表

指标	目标	当前值	状态	数据来源
不破句率	100%	100%	已达标	statistics.py 实测 (3文档)
表格/公式/代码整体成块率	≥95%	100%	已达标	statistics.py 实测 (3文档)
目标长度区间命中率	≥90%	94.4%	已达标	statistics.py 实测 (3文档)
原文回链完整率	100%	100%	已达标	statistics.py 实测 (3文档)
Recall@5 提升 (Smart vs Fixed)	≥+10%	-12.7%	未达标	eval_rag.py 实测 (3文档×15题)
nDCG@5 提升	相对 baseline 提升	-20.6%	待改善	eval_rag.py 实测
MRR 提升	相对 baseline 提升	-18.9%	待改善	eval_rag.py 实测
标签准确率	≥85%	待评测	待评测	7月3日运行
摘要忠实度	≥90%	待评测	待评测	7月3日运行

13 里程碑验收状态

13.1 M1 — 架构与接口冻结（6月24日）

验收项：

DocumentNode、Chunk、SegmentConfig schema 确定

API 接口列表确定 (/health、/segment、/organize、/evaluate、/strategies)

目录结构和技术栈确定

样例数据确定 (3 份文档)

状态：已通过

13.2 M2 — 基础分段可运行（6月27日）

验收项：

/segment 可运行

能输出基础 chunk (含 content、char_count)

能输出 title_path

能输出 source_refs

表格/公式/代码保护生效

长度控制生效 (过长拆分、过短合并)

状态：已通过

13.3 M3 — 核心智能体闭环初步可用（6月30日）

验收项：

语义边界增强已集成

特殊块保护完善 (5 类)

摘要生成（规则 + LLM 双模式）

关键词/主题标签生成

实体标签生成

回链元数据完整（title_path + source_refs + strategy_info + quality_flags）

/organize 可运行

ContentOrganizer 编排器完成

LLM 降级链路完成

文档级摘要聚合完成

状态：已通过（10/10）

13.4 M4 — 对比实验结果可输出（目标：7月3日）

验收项：

- ☒ fixed_length baseline 可运行 + 对比结果可输出
- ☐ heading_based baseline 可运行（7月3日实现）
- ☒ Recall@k 可输出
- ☒ nDCG 可输出
- ☒ MRR 可输出
- ☐ 三策略完整对比表（7月3日完成）
- ☐ 最优默认策略参数确定（7月3日完成）

当前进度：约 75%（5/7 项完成）

状态：进行中，目标 7月3日验收

13.5 M5 — 验收候选版本（目标：7月6日）

验收项：

- ☐ FastAPI 全链路联调通过
- ☐ 命令行脚本完善
- ☐ 日志和配置文件就绪
- ☐ 单元测试 ≥ 20 个通过
- ☐ 接口测试通过
- ☐ 样例回归通过
- ☐ 演示页面/报告准备完成

状态：待进行（7月4日开始）

13.6 M6 — 最终交付包冻结（目标：7月9日）

验收项：

- ☐ README 完整
- ☐ API 文档完整
- ☐ UI 文档完整

- ☐ 评测报告完整
- ☐ 演示材料完整
- ☐ 代码冻结（不新增功能）
- ☐ 数据冻结（评测数据、输出样例）
- ☐ 提交包结构符合计划

状态：待进行（7月7日开始）

13.7 M7 — 最终提交与验收（7月10日）

验收项：

- ☐ 最终提交包完整
- ☐ 现场演示成功
- ☐ 老师问题回答
- ☐ 后续优化方向记录

状态：待进行

14 风险管理与缓冲消耗

14.1 已识别风险

风险编号	风险描述	影响	概率	当前状态	应对措施
R-1	embedding 模型下载失败	语义分段和检索评测受阻	低	已缓解	降级方案（字符3-gram）已就绪，当前 MiniLM 正常工作
R-2	LLM API 不可用	内容组织质量下降	中	已缓解	规则模式兜底已实现，ContentOrganizer 自动降级
R-3	Recall@5 无法达到 +10% 目标	核心验收指标不达标	中	密切监控	参数调优（7月3日）→ 根因分析（已完成）→ 向老师汇报 → 技术报告解释
R-4	评测数据不足导致指标不稳定	评测结论不可靠	中	监控中	明天扩充至 30 题，分层报告增强统计稳定性
R-5	组员时间冲突（考试/课程）	进度延误	低	正常	备份机制就绪，tune 搜索可自动运行
R-6	老师反馈导致返工	进度延误	低	未触发	计划 7月3日主动向老师同步进展
R-7	前端演示页面来不及	演示效果打折扣	低	正常	静态 Markdown 报告降级方案已就绪
R-8	集成测试发现关键 bug	返工耗时	中	未触发	7月4-6日集中测试，预留缓冲时间

14.2 缓冲消耗情况

缓冲类型	计划缓冲	已消耗	剩余	消耗率
阶段1+2 缓冲	~10h	~5h	~5h	50%
阶段3 缓冲	~15h	~18h	-3h	120%（已超额 3h）
预留风险缓冲	18h	0h	18h	0%
总缓冲	43h	23h	20h	53.5%

缓冲消耗分析：

- 阶段3 语义边界调试 (+5h) 和内容组织评测标注 (+3h) 超出了阶段3的 15h 缓冲 (实际消耗 18h, 超额 3h)
- 预留风险缓冲 18h 尚未动用, 可用于覆盖后续阶段的意外耗时
- 阶段5 (集成测试) 和阶段6 (文档撰写) 通常是返工高发期, 需要保留至少 10h 缓冲
- 当前总缓冲剩余 20h, 仍处于安全区间

14.3 降级方案状态

风险模块	首选方案 (当前)	降级方案	降级触发条件
Embedding 模型	MiniLM (384维, sentence-transformers)	字符 3-gram (256维, CRC32哈希)	sentence-transformers 导入/加载失败
内容组织	LLM (DeepSeek/OpenAI)	规则 (jieba TF-IDF + 正则)	API Key 未配置或调用失败
前端演示	Vue 3 页面	静态 Markdown/HTML 报告	7月6日仍未完成前端集成
向量检索	numpy EmbeddingStore	ChromaDB (项目已集成)	评测规模增长至 50+ 文档
评测数据集	15 题 (今日) / 30 题 (明天)	维持 15 题 + 详细 per-question 分析	标注人力不足或时间不够

15 附录：关键代码与数据

15.1 附录 A：项目目录结构

```
1 C:\Users\yh200\Desktop\zhinengti/
2 |— README.md # 项目说明 (已更新至6月30日状态)
3 |— CHANGELOG.md # 变更日志
4 |— pyproject.toml # Python 项目配置
5 |— assets/
6 |   |— title.md # 评测文档1: 课题说明
7 |   |— 同类开源方案调研报告.docx # 评测文档2: 调研报告
8 |   |— 开题报告_课题11_面向RAG的智能分段与内容组织智能体.docx # 评测文档3: 开题报告
9 |   |— 研发计划.pdf # 研发计划 (今日参考)
10 |— backend/
11 |   |— requirements.txt # Python 依赖
12 |   |— app/
13 |       |— main.py # FastAPI 入口
14 |       |— api/routes.py # REST API 端点
15 |       |— core/config.py # 全局配置 (含 LLM 配置)
16 |       |— models/schemas.py # Pydantic 数据模型
17 |       |— services/
18 |           |— embedding.py # Embedding 编码器 + 降级 (124行)
19 |           |— segmenter.py # 对外统一导出
20 |           |— organizer.py # 内容组织编排器
21 |           |— model_client.py # RuleBasedTagger + LLMClient
22 |           |— evaluator.py # 标签/摘要/实体质量评估
23 |           |— document_loader/ # 多格式文档加载 (txt/md/docx/pdf)
24 |           |— document_preprocessor.py # 封面/目录去除
25 |           |— segmenting/ # 核心分段引擎
26 |               |— models.py # DocumentBlock, Chunk, SegmentConfig
27 |               |— parser.py # 纯文本解析
28 |               |— heading.py # 标题识别与层级估计
29 |               |— splitter.py # 候选分段/超长拆分/过短合并
30 |               |— segmenter.py # 主流程编排
31 |               |— enrichment.py # 分段富化
32 |               |— keyword_extraction.py # 关键词提取
```

```

33 |         | └─ statistics.py      # 统计指标 (133行)
34 |         | └─ organizer/        # 内容组织模块
35 |         |   └─ organizer.py    # ContentOrganizer (~120行)
36 |         |   └─ model_client.py # RuleBasedTagger + LLMClient (~200+行)
37 |         |   └─ __init__.py
38 |         | └─ evaluation/       # RAG 评测框架
39 |         |   └─ baseline.py     # fixed_length baseline (93行)
40 |         |   └─ metrics.py     # IR 指标 + 相关性判定 (275行)
41 |         | └─ retrieval/       # 检索后端
42 |         |   └─ embedding_store.py # numpy 向量检索 (230行)
43 |         |   └─ tfidf_store.py  # TF-IDF 检索
44 |         |   └─ semantic_store.py # 接口兼容层
45 |         | └─ rag_store/       # RAG 持久化存储
46 |         |   └─ service.py     # 存储服务编排
47 |         |   └─ chroma_store.py # ChromaDB 向量存储
48 |         |   └─ sqlite_store.py # SQLite 元数据
49 |         | └─ preprocessing/   # 文档预处理
50 |         |   └─ cleaner.py     # 封面/目录/表格清洗
51 | └─ backend/tests/
52 |   └─ test_segmenting.py       # 分段引擎测试 (8 tests)
53 |   └─ test_keyword_extraction.py # 关键词提取测试 (5 tests)
54 |   └─ test_query_retrieval.py  # 检索功能测试
55 |   └─ eval_dataset.py         # 评测数据集 (119行, 3文档×5题)
56 | └─ scripts/
57 |   └─ segment_file.py         # CLI 分段工具
58 |   └─ eval_rag.py             # RAG 评测主脚本 (228行)
59 |   └─ tune_segmenting_params.py # 参数网格搜索 (136行)
60 |   └─ human_eval_semantic.py  # 语义完整性人工评估 (299行)
61 |   └─ synthesize_qa_pairs.py  # QA 对合成 (429行)
62 |   └─ draw_architecture.py    # 架构图生成
63 | └─ frontend/                # Vue 3 前端
64 |   └─ src/
65 |     └─ App.vue
66 |     └─ views/HomeView.vue
67 |     └─ components/
68 |       └─ ConfigPanel.vue
69 |       └─ ChunkList.vue
70 |       └─ MetricPanel.vue
71 |       └─ RetrievalPanel.vue
72 |     └─ api/chunking.js
73 |     └─ stores/chunkStore.js
74 |     └─ router/index.js
75 | └─ data/
76 |   └─ outputs/                # CLI 输出目录
77 |   └─ rag/                    # RAG 存储数据
78 |     └─ chroma/              # ChromaDB 持久化索引
79 |     └─ rag_meta.db          # SQLite 元数据
80 | └─ 日报_2026-06-26.md
81 | └─ 日报_2026-06-29.md
82 | └─ 日报_2026-06-30.md
83 | └─ 日报_2026-07-02.md      # 本文件

```


15.2 附录 B: 评测数据集 (eval_dataset.py) 完整内容概要

```
1 共 3 份文档, 15 个问题
2
3 文档1: eval_title (assets/title.md)
4   Q1: 这个课题的名称是什么?
5       answer_keywords: [课题11, 面向RAG, 智能分段, 内容组织, 智能体]
6   Q2: 分段策略中需要保护哪些特殊内容块?
7       answer_keywords: [表格, 公式, 代码, 整体成块, 不截断]
8   Q3: 验收标准中不破句率和目标长度命中率的要求是多少?
9       answer_keywords: [不破句率, 100%, 目标长度区间命中率, 90%]
10  Q4: 这个课题与朴素固定长度切分的区别是什么?
11       answer_keywords: [结构, 语义感知, 闭环评估, 下游检索]
12  Q5: 课题要求交付哪些东西?
13       answer_keywords: [可运行, 分段智能体, 标准接口, chunk序列, 对比实验]
14
15 文档2: eval_survey (assets/同类开源方案调研报告.docx)
16   Q1: 调研报告分析了哪些开源分段工具?
17       answer_keywords: [LangChain, LlamaIndex, Unstructured, Docling]
18   Q2: 开源方案在语义分段方面有什么不足?
19       answer_keywords: [固定长度, 分隔符, 语义感知弱, 标题层级]
20   Q3: 调研中提到了哪些检索评估指标?
21       answer_keywords: [Recall, nDCG, MRR, 命中率]
22   Q4: 哪些项目支持多模态文档解析?
23       answer_keywords: [Docling, Unstructured, PDF, 图像]
24   Q5: 报告中对RAGFlow的评价是什么?
25       answer_keywords: [RAGFlow, 元数据, 溯源, chunk]
26
27 文档3: eval_open_report (assets/开题报告_课题11_面向RAG的智能分段与内容组织智能体.docx)
28   Q1: 开题报告中描述的系统总体目标是什么?
29       answer_keywords: [设计并实现, 面向RAG, 智能分段, 内容组织, 智能体]
30   Q2: 报告中提到了哪些技术难点?
31       answer_keywords: [语义完整, 边界不破坏, 长度可控, 检索指标, 闭环验证]
32   Q3: 系统的验收标准是什么?
33       answer_keywords: [不破句率, 整体成块率, 命中率, 完整率]
34   Q4: 技术方案中采用了什么架构?
35       answer_keywords: [FastAPI, Vue, Python, 前端, 后端]
36   Q5: 报告中提到了哪些创新点或差异化优势?
37       answer_keywords: [结构感知, 语义感知, 闭环评估, 下游检索, 优化目标]
```

15.3 附录 C: Smart 策略当前默认参数

```
1 SegmentConfig(
2     min_chars=300,                # 最小 chunk 字符数
3     target_chars=900,            # 目标 chunk 字符数
4     max_chars=1200,              # 最大 chunk 字符数
5     min_tokens=150,              # 最小 token 数
6     target_tokens=450,           # 目标 token 数
7     max_tokens=600,              # 最大 token 数
8     heading_flush_min_chars=240, # 标题边界刷新阈值
9     overlap_sentences=1,         # 重叠句子数
10    include_heading_in_content=True, # chunk 内容是否包含标题
11    enable_semantic_boundary=True,  # 启用语义边界
12    semantic_boundary_threshold=0.45, # 语义边界相似度阈值
13    keyword_strategy="jieba",      # 关键词提取策略
14    recursive_separators=["\n\n", "\n", "\t", "!", "?", ":", ";", ",", "!", "?", ";", "."],
```

```
15 |         table_split_threshold=1500,          # 超长表格拆分阈值
16 |     )
```

15.4 附录 D: EmbeddingRelevance 相关性判定逻辑

```
1 | def is_relevant(self, chunk_content: str) -> bool:
2 |     # 步骤1: 关键词命中率 >= 34% (约 1/3 的关键词命中)
3 |     keyword_score = self.keyword_score(chunk_content)
4 |     if keyword_score >= 0.34:
5 |         return True
6 |
7 |     # 步骤2: 有关键词命中 + chunk 包含数字/指标 → 宽松判定
8 |     if keyword_score > 0 and contains_number_or_metric(chunk_content):
9 |         return True
10 |
11 |     # 步骤3: 语义相似度判定
12 |     # 有关键词时提高阈值 (0.57) , 防止纯语义假阳性
13 |     semantic_threshold = max(self.threshold, 0.57) if self._keywords else
self.threshold
14 |     return self.score(chunk_content) >= semantic_threshold
```

15.5 附录 E: 今日评测运行环境

项目	信息
Python 版本	≥ 3.12
sentence-transformers	≥ 3.0.0
Embedding 模型	paraphrase-multilingual-MiniLM-L12-v2 (118MB, 384维)
向量检索	numpy 余弦相似度 (L2归一化 + np.dot)
评测文档	3 份 (1 MD + 2 DOCX)
评测问题	15 题
分段策略	Smart (heading_recursive_semantic_rule) 、Fixed (512字符窗口+句子边界)